National University of Singapore

Institute of Systems Science
Intelligent Reasoning Systems
Project report

GROUP PROJECT REPORT

AnchorAI

Explainable, budget-aware retention planning for gyms

Churn prediction · uplift estimation · rule-based reasoning · case-based reasoning
genetic algorithm with an exact CP-SAT benchmark · knowledge graph · grounded staff assistant

TEAM

Abdulaziz Ali S Hasanain	A0350856X
Asiri Mohammed Yahya A	A0351967N
Binmaneea Khalid Waleed M	A0355712E

National University of Singapore

Group 16 · 25 October 2026

Abstract

About one gym member in three leaves each year, and most stop attending weeks before they cancel. Retention tools flag members at risk but leave the decision that matters (whom to contact this week, with which offer, within a fixed budget and limited staff hours) to guesswork. AnchorAI makes that decision and explains it. A calibrated ensemble predicts churn on real members; a forward-chaining rule base encodes the gym's playbooks and guardrails; case-based reasoning estimates what each offer would achieve for each member; a genetic algorithm, benchmarked against an exact CP-SAT solver, builds the weekly plan; a knowledge graph adds signals the churn model cannot see, such as a departed instructor; and a grounded assistant answers staff questions from the system's own evidence. Every recommendation carries its reasons, and managers approve, override and audit the plan in one web application.

Segmentation and churn prediction are evaluated on 4,000 real members of a public dataset. Decisions are evaluated on a documented synthetic scenario layer whose true offer effects are known, so plans can be graded against a truth no planner sees; their gains show that the method works when effects are of this kind, not how large they are in a real gym.

0.908
churn F1 on 800 unseen real members (PR-AUC 0.973)

+22–138%
more members kept than contacting by risk or by schedule, at equal budget

0.24%
largest gap of the genetic algorithm to the proven optimum

96.2%
of the assistant's judged sentences supported by its evidence

Keywords churn prediction · uplift modelling · case-based reasoning · genetic algorithms · constraint programming · rule-based systems · knowledge graphs · retrieval-grounded assistants

Contents

1 Business case, market research and objectives 1
1.1 The business problem . . . 1
1.2 Market and positioning . . . 1
1.3 Pricing and business value . . . 2
1.4 Target user and approach . . . 2
1.5 Objectives and success measures . . . 3

2 System design and architecture 5
2.1 Components . . . 5
2.2 How the parts connect . . . 5
2.3 The weekly cycle . . . 6
2.4 Technology and design principles . . . 6

3 Data and the scenario layer 8
3.1 Source data . . . 8
3.2 Preparation and features . . . 8
3.3 The scenario layer . . . 9
3.4 Data contract and reproducibility . . . 11

4 Member intelligence: segments, churn and uplift 12
4.1 Behavioural segmentation . . . 12
4.2 Churn model . . . 13
4.3 Uplift baseline . . . 13

5 Decision automation: rules and knowledge acquisition 15
5.1 The rule engine . . . 15
5.2 The rule base . . . 15
5.3 Knowledge acquisition . . . 16
5.4 Administration and explainability . . . 17

6 Decision and optimisation: CBR, GA and CP-SAT 18
6.1 The planning problem . . . 18
6.2 Case-based reasoning . . . 18
6.3 Genetic algorithm . . . 19
6.4 CP-SAT exact planner and benchmark . . . 20
6.5 Baselines, evaluator and the path to action . . . 20

7 Knowledge graph and staff assistant 21
7.1 Knowledge graph . . . 21
7.2 Staff assistant . . . 22

8 System development and implementation 24
8.1 From raw file to served scores . . . 24
8.2 The application . . . 25

8.3 Degraded operation . . . 25
8.4 Testing and quality assurance . . . 26

9 Experiments and findings 27
9.1 E1: Are the segments meaningful? . . . 27
9.2 E2: Does the churn model beat simple rules? . . . 28
9.3 E3: Can we find who is persuadable? . . . 29
9.4 E4: Does AnchorAI retain more members per dollar? . . . 30
9.5 E5: How good and how fast is the genetic algorithm? . . . 31
9.6 E6: Are the rules correct, and do guardrails matter? . . . 31
9.7 E7: Do knowledge-graph signals add information? . . . 32
9.8 E8: Is the assistant accurate and grounded? . . . 33
9.9 Summary of findings . . . 34

10 Findings, discussion and future work 35
10.1 What we built and what we found . . . 35
10.2 Limitations . . . 35
10.3 Recommendations for an operator . . . 36
10.4 Future work . . . 36

References 37

A Project proposal 39

B Mapping to MR, RS and CGS 43

C Scenario layer parameters 45
C.1 Tables and parameters . . . 45
C.2 Ground truth . . . 46
C.3 Limitations of the scenario . . . 47

D Installation guide 48

E User guide 50
E.1 Manager: the weekly cycle . . . 50
E.2 Staff: contacts, drafts and the assistant . . . 51
E.3 Administrator . . . 52

F Use of AI tools 54

G Individual project reports 57
Abdulaziz Ali S Hasanain . . . 57
Asiri Mohammed Yahya A . . . 58
Binmaneea Khalid Waleed M . . . 59

1 Business case, market research and objectives

1.1 The business problem

Gym members rarely cancel on impulse. They stop coming first, and cancel weeks or months later; that lag is the window in which a well-chosen contact can still keep them. The window is valuable. The Health & Fitness Association puts the median member retention rate at 66.4% in 2024, across 175 operators running more than 17,000 facilities [1]: about one member in three leaves each year, taking dues and ancillary spend with them. New members are the most fragile. In a study of one fitness centre, 63% stopped before their third month and fewer than 4% stayed continuously active beyond twelve [2], and the real members analysed in this report agree: the newest segment churns at 94.6% (Section 9.1).

The proposal identified three ways in which current practice fails (Table 1.1). The common thread is that each produces either a list or a score, but not a decision: none says whom to contact this week, with which offer, within a fixed budget and finite staff hours, and why.

1.2 Market and positioning

Memberships grew 6% and facilities nearly 4% in 2024 [3], so there are ever more members to keep. Club software divides into systems of record (membership, booking, payments, access control; for example ABC Fitness, Mindbody, Xplor, Perfect Gym, Virtuagym and Daxko) and an intelligence layer that bolts onto them, of which Keepme, with its predictive member score, is the clearest example; Glofox offers a built-in at-risk report. The proposal's desk research, based on vendor material and buyer guides that have a commercial interest, found four gaps:

1. **Scores, not allocation.** No vendor publicly exposes constrained allocation: which flagged members to contact, with which offer, under a budget and staff hours.
2. **Risk, not response.** Vendors target the members most likely to leave, but on field data, targeting by the expected response to the intervention reduces churn more than targeting by risk [4].
3. **Opaque scores.** Models trained on very large member pools are powerful but unauditable, and mid-market operators cite staff distrust of black-box flags.
4. **Priced for chains.** The intelligence layer is generally recommended from five locations upwards, which leaves the one-to-five-club operator underserved.

Table 1.1 Current retention practice and its failure modes, as identified in the proposal.

Practice	Why it fails
Calendar campaigns	Contact everyone on a schedule, so budget goes to members who would have stayed anyway.
Threshold rules	Ignore trend: twelve visits a month falling to four is a stronger warning than a steady two, but a threshold treats them alike.
Vendor risk scores	Give a number, not a decision; staff cannot interrogate the flag, so they ignore it.

AnchorAI is therefore positioned as an *explainable, budget-aware retention planner for mid-market operators*. It does not try to out-predict incumbents with far larger training data; it differentiates on the decision and on its transparency. We have not tested any competitor product, so every comparison in this report is against baseline policies in our own experiments, not against a vendor.

1.3 Pricing and business value

AnchorAI is sold as an annual subscription that bolts onto the operator's existing membership system: US$199 a month for the first club and US$149 for each further club up to five, so a three-club operator, the size of our scenario, pays US$497 a month. It runs on the operator's own machine, language model included: in E8 a local seven-billion-parameter model was as faithful to its evidence as the hosted one (95.1% of judged sentences supported for each), so there is no fee per question. A hosted and maintained option would cost more, to cover hosting and support; we have not costed it.

The price sits where the gap in the market is (Table 1.2). The dedicated retention layer is priced for chains: in 2020 Keepme's founder quoted US$9,000 a year for a single site, with a lower price for each further site [5], about US$750 a month, nearly four times AnchorAI's first club. Small operators already pay at AnchorAI's level for their membership system and its add-ons [6, 7].

Table 1.2 AnchorAI's price against published prices of club software, in US dollars a month.

Product	What it is	Price a month
AnchorAI	retention planner: first club; each further club, up to five	199; 149
Keepme	retention intelligence layer, single site (2020 figure) [5]	about 750
PushPress	membership system; marketing add-on [6]	159–229; 329
Gymdesk	membership system by member count; each extra location [7]	75–240; 50–100

The value comes from members kept. The planner values a kept member at six months of fees, a deliberately cautious horizon; at the scenario's middle fee of US$54, for a six-month contract, that is US$324, an assumption of the scenario layer rather than a market figure. One club therefore breaks even when AnchorAI keeps 0.6 members a month who would otherwise have left, and three clubs need 1.8. In the scenario, at the default 36 staff hours and US$1,500, a week's plan from the greedy planner on the case-based estimates keeps 21.0 members against 13.7 for contacting by risk, and spends US$432 instead of US$1,280 (E4): 7.3 more members from a single week, four times the monthly break-even for three clubs. The scenario's effects are synthetic, so this shows that the margin is wide, not how wide it would be in a real gym. The direction agrees with field evidence that targeting by expected response reduces churn more than targeting by risk [4], and the stake is large, since about a third of members leave each year [1].

1.4 Target user and approach

The target user is a gym business with one to five clubs, and the people who use AnchorAI are its club managers, front-desk staff and trainers. No real operator was involved: we had no operator's data, and the manager interviews used to shape the rule base were simulated and are treated as plausible expert opinion, not evidence (Section 5.3).[1]

[1]The proposal named the team's study sponsor; it is not a client or data provider of this project and has not reviewed the system.

Table 1.3 Objectives.

	Objective	Why it matters to a gym
O1	**Know who is at risk, and why**: interpretable segments and a calibrated churn probability with its drivers.	Staff act on reasons, not on a bare score.
O2	**Know whom a contact would keep**: the extra chance a member stays if offered each offer (its *uplift*).	The members most likely to leave are not always the ones a call can save.
O3	**Plan the week within its limits**: who to contact, with which offer and channel, within budget, staff hours, stock and contact limits.	This is the decision a manager actually makes, and the one current tools leave open.
O4	**Keep people in control**: every recommendation explained and checked against the gym's rules; overrides with a reason; approval; a full audit trail.	Staff distrust black-box flags; a plan they can question is one they will use.
O5	**Make it usable by gym staff**: one web application, and an assistant that answers in plain language from the system's own data.	Front-desk staff and trainers are not analysts.
O6	**Make it dependable**: reproducible results, and planning that continues when the knowledge graph or language model is down.	A tool that fails with one service will not be trusted with a weekly routine.

AnchorAI answers the weekly question with one pipeline (Chapter 2). It learns from real members who is at risk and why (Chapter 4); checks the gym's playbooks and guardrails with a rule engine (Chapter 5); estimates what each offer would achieve for each member and chooses the best plan within the week's limits (Chapter 6); adds signals from a knowledge graph of who trains with whom; and lets staff question every recommendation in plain language (Chapter 7). Managers approve the plan, and every change is audited (Chapter 8).

1.5 Objectives and success measures

The proposal set three goals: segment members interpretably, predict churn with a calibrated ensemble, and turn the predictions into a weekly action plan whose every recommendation carries a justification staff can challenge. Building the system showed that the third goal needs more than a churn score. A plan has to know which members a contact would actually keep, respect the week's budget and staff time, and leave the final decision with people. We therefore pursued six objectives (Table 1.3) and attached to each a measure, a baseline or target, and an experiment (Table 1.4).

Scope of the evaluation. The business result is measured on a synthetic gym. The real data has no offers and no record of who was contacted, so no real outcome of a plan exists; the scenario layer (Section 3.3) gives every member a known response to every offer, which lets plans be graded fairly, while the size of any gain depends on how that response was designed. Explainability and usability are assessed by design and through the assistant's evaluation, not by user testing: no gym staff used the system, and the manager interviews were simulated (Section 5.3).

Table 1.4 Success measures and verdicts. The verdicts are argued in Chapter 9.

	Exp.	Measure	Baseline or target	Verdict
O1	E1	segment stability across seeds; churn separation	other k from 2 to 10	● Met
O1	E2	churn F1, PR-AUC, Brier; calibration by band	visit-count rule; logistic regression	● Met
O2	E3	rank correlation of estimated with true uplift	T-learner; risk as uplift; random	● Partly met
O3	E4	extra members kept per $1,000 and per staff hour; feasibility	contact-all; risk-top-k	● Met
O3	E5	optimality gap and run time of the GA	CP-SAT optimum; 2,000 members in 30 s	● Partly met
O4	E6	rule-base consistency; plan value with and without guardrails	expected cases; no guardrails	● Met
O1	E7	information added by knowledge-graph signals	churn probability alone	● Partly met
O5	E8	intent accuracy; faithfulness of answers	keyword router	● Partly met
O4, O6	tests	explanation and audit coverage; degraded operation	every item explained, every write audited	● Met

2 System design and architecture

In brief

- One web application and one API serve three roles; behind them, three reasoning components each use the techniques that suit their job.
- The components meet at fixed contracts: shared data tables, an allowed-offer list from the rules, and one explanation record per plan item.
- A weekly cycle runs from scores to an approved, assigned call list, and the logged outcomes feed the next week.

AnchorAI turns one weekly question, *which members should we contact this week, with which offer, within our budget and staff hours?*, into an approved call list that every staff member can act on and every manager can question.

2.1 Components

The system has four layers (Figure 2.1). A single web application serves administrators, managers and staff. One FastAPI service checks every request's role, audits every change and runs long jobs in the background. Three reasoning components do the work (Table 2.1), and four stores hold the state: versioned *data tables* rebuilt from code, an *application database* for everything users create (plans, assignments, outcomes, rule versions, offers, roster, audit log), the *Neo4j* knowledge graph, and any OpenAI-compatible *language model*, local or hosted.

2.2 How the parts connect

Three contracts keep the components independent. *The data tables* are written once by the data build and read by everything else; their columns are fixed in code and checked on every build. *The allowed-offer list* is the rule engine's output: for each member it reads scores, features, recent contacts and knowledge-graph signals, and returns the permitted offers and channels with the trace of rules that fired. Guardrails are therefore hard constraints, because the optimiser never sees an offer the rules block. *The explanation record* travels with every plan item: churn probability and drivers, rule

Table 2.1 The three reasoning components.

Component	Job	Techniques
Member intelligence (Chapters 3 and 4)	Prepare the real members, attach the synthetic gym, score every member	K-means, calibrated logistic regression and LightGBM ensemble, per-member drivers, T-learner baseline
Knowledge and interaction (Chapters 5 and 7)	Encode the gym's policies and relationships; answer staff questions	forward-chaining rules with guardrails and playbooks, Neo4j knowledge graph, intent-routed grounded assistant
Decision and optimisation (Chapter 6)	Estimate each offer's effect and choose the best plan within the week's limits	case-based reasoning, genetic algorithm, CP-SAT, greedy baseline, policy evaluator

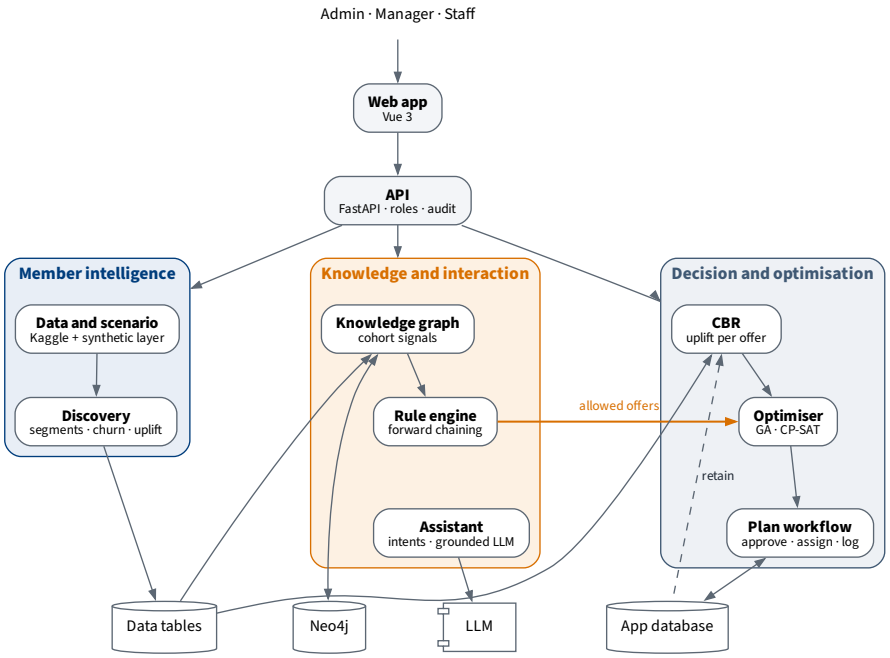


Figure 2.1 Architecture. Arrows show the direction of data flow; double-headed arrows read and write; the dashed arrow is the learning loop through which logged outcomes return to the case base.

trace and graph signals, the ten most similar past cases and what happened to them, and a one-line summary. The plan's 'why' panel and the assistant's answers read the same record, so both always give the same reason.

2.3 The weekly cycle

Figure 2.2 shows one week. Steps 1–4 score members, list their allowed offers, estimate each offer's uplift and build the plan in the background with the chosen planner: the genetic algorithm by default, exact CP-SAT, or greedy. In step 5 the manager opens 'why' on any member, removes or adds contacts with a reason (the rules are re-checked at once) and can re-optimise the remaining budget; approval locks the plan, one per week. Approval shares the calls among staff by weekly hours and skills, sending training sessions to trainers (step 6), and staff work through their calls in priority order with talking points and an editable message draft, logging each outcome in two taps (step 7).

Outcomes feed the next week twice. Recent contacts count towards the contact limits, so no member is contacted again too soon, and accepted or declined offers become new cases for the reasoner. These logged cases serve as evidence only: the average effect of each offer is still taken from the randomised history, because outcomes from the planner's own choices would bias it.

2.4 Technology and design principles

Seven principles shaped the design (Table 2.2 lists the stack).

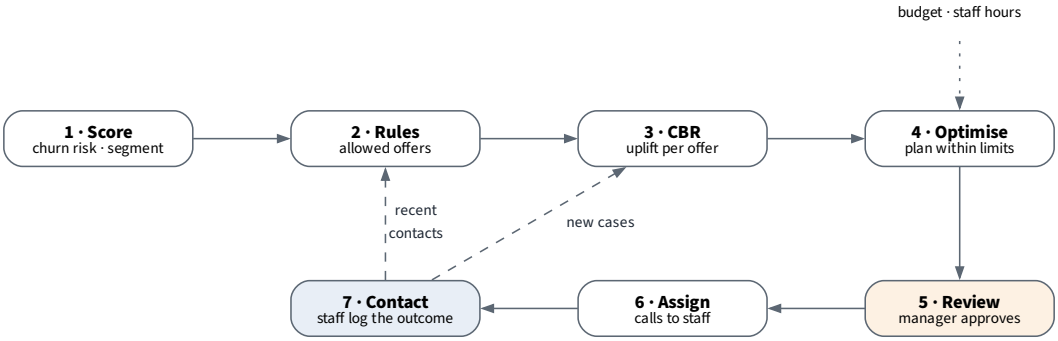


Figure 2.2 The weekly cycle. Once the manager sets the week's budget and staff hours, steps 1–4 run automatically; the manager then reviews and approves (orange), and staff contact members and log outcomes (blue). Dashed arrows carry this week's outcomes into next week's plan.

Table 2.2 Technology.

Layer	Technology
Web application	Vue 3, TypeScript, Vite, PrimeVue, ECharts, Cytoscape.js, Pinia
API and storage	Python 3.12, FastAPI, JWT sign-in with bcrypt; SQLite via SQLModel with Alembic migrations and deterministic seeds
Data and models	pandas, scikit-learn, imbalanced-learn, LightGBM, SHAP
Optimisation	DEAP genetic algorithm [8], OR-Tools CP-SAT [9]
Knowledge and language	Neo4j 5 with parameterised Cypher; sentence-transformers for intents; OpenAI-compatible LLM client

- **People decide.** Nothing reaches a member without a manager's approval and a staff member's call; message drafts are never sent automatically.
- **Explainable by construction.** Every recommendation carries the rules that allowed it, the precedents behind it and the drivers of the member's risk.
- **Role-based access, enforced twice.** The interface hides what a role cannot use, and the API checks the role again on every request.
- **Every change audited.** Approvals, overrides, assignments, outcomes and rule or user edits record who, when, and the value before and after.
- **Reproducible.** Every random process takes a seed; data, models and demo data are rebuilt from code; 195 offline tests run in under a minute.
- **Graceful degradation.** Without Neo4j the rules use the graph facts cached at the last load; without the language model the assistant returns the same cited evidence in a template. The planner needs neither.
- **One component, one job.** Each component sits behind its own routes and agreed formats, so it can be tested and replaced on its own.

3 Data and the scenario layer

In brief

- Segmentation and churn prediction use only the 4,000 real members of a public Kaggle dataset.
- The snapshot has no clubs, classes, offers or contact history, so a documented synthetic *scenario layer* supplies them, together with a ground truth of how each member responds to each offer.
- Every table, model and score is rebuilt from the raw file by one deterministic command.

3.1 Source data

AnchorAI is built on the public dataset *Gym Customers Features and Churn* [10], which is cited and not redistributed. It describes 4,000 members of a gym chain in 14 columns, with no missing values and no duplicate rows. The label `churn` marks the 1,061 members (26.5%) who left. Because a model that predicts 'stays' for everyone is already 73.5% accurate while catching no churner, accuracy is never used; the experiments report F1 on the churn class, PR-AUC and the Brier score instead.

The data is a single snapshot: one row per member, without dates, clubs, classes, instructors, offers or past retention contacts. This one fact shapes the rest of the design. What can be learned from real members (who is at risk, and why) is learned from them; what cannot (how a member responds to an offer, who attends which class) comes from the scenario layer in Section 3.3, and every experiment states which of the two it uses.

3.2 Preparation and features

Each member gets a pseudonymous identifier (`M0001` to `M4000`). A split stratified on churn gives 3,200 training members and 800 test members, both with a 26.5% churn rate. Nothing is ever fitted on the test labels: a regression test flips every one of them and checks that no score changes. Two synthetic columns complete the member record: a club (one of three, at random) and a monthly fee by contract length ($72, $54 or $46 for 1, 6 or 12 months), an assumption used to value a retained member.

Table 3.1 The Kaggle columns, over all 4,000 members.

Group	Columns	Values
Profile	gender, age, near location, phone	age 18–41 (mean 29.2); 85% live or work nearby; 90% have a phone on file
Membership	contract period, months to end, lifetime, partner, promo friends	contracts of 1, 6 or 12 months (median 1); median lifetime 3 months; 49% via a partner company; 31% referred by a friend
Activity	group visits, weekly visits (lifetime and this month)	41% attend group classes; mean 1.88 visits a week over the membership, 1.77 this month
Spend	additional charges	café, goods and services; mean 147
Target	churn	1 = left; 26.5%

Source: `a_features_leakage.ipynb`, cell 2.

Table 3.2 Derived features.

Feature	Definition	Intent
`trend_ratio`	this month's weekly visits ÷ lifetime weekly visits, capped at 3	attendance falls before a member cancels
`visit_change`	this month's minus lifetime weekly visits	the same signal on an absolute scale
`contract_progress`	share of the contract already used	members near the end of a contract are free to leave
`charges_per_month`	extra spend ÷ (months as a member + 1)	spend intensity, not accumulated spend

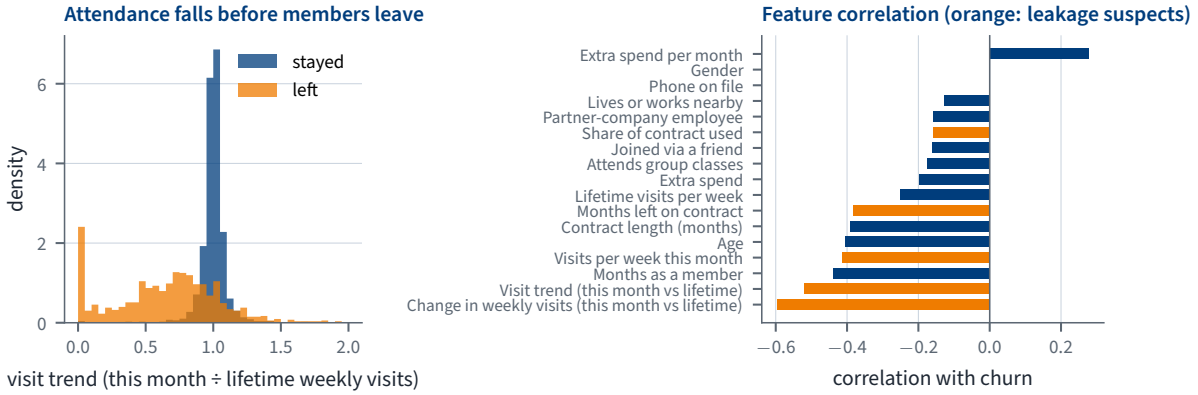


Figure 3.1 Left: the visit trend of members who stayed clusters at 1, while members who left spread below it, with a spike at 0 for those who did not come this month (medians 1.00 and 0.69). Right: correlation of each feature with churn; the two strongest are leakage suspects. Real Kaggle data.

Source: `scripts/figures.py` (`trend`), as in `a_features_leakage.ipynb`, cells 2–4.

The churn model reads the 13 Kaggle predictors and four features derived from them (Table 3.2). The proposal planned seven feature families; the snapshot supports five of them, while recency and regularity would need visit dates it does not have.

Leakage. The snapshot has no cancellation-process fields (freeze requests, removed payment methods), which the proposal warned against. Its risk is timing instead: this month's visits and the months left on the contract may be measured in the same month as the label. These two columns and the three features built from them form a *suspect set* that includes the strongest single signals. The served model keeps them, because both values are known when a weekly plan is made, but E2 reports the model with and without them so that the honest lower bound is visible (Section 9.2).

3.3 The scenario layer

The knowledge graph, the case-based reasoner, the uplift models and the optimiser all need what the snapshot lacks. Rather than an agent-based simulator, as proposed, a thin synthetic gym is attached to the real members, and its parameters are listed in Chapter C. Its purpose is to make decisions gradable: because every member's true response is known, a plan can be scored against a truth that no planner sees. Results built on it show that the system recovers known structure, not how real members behave.

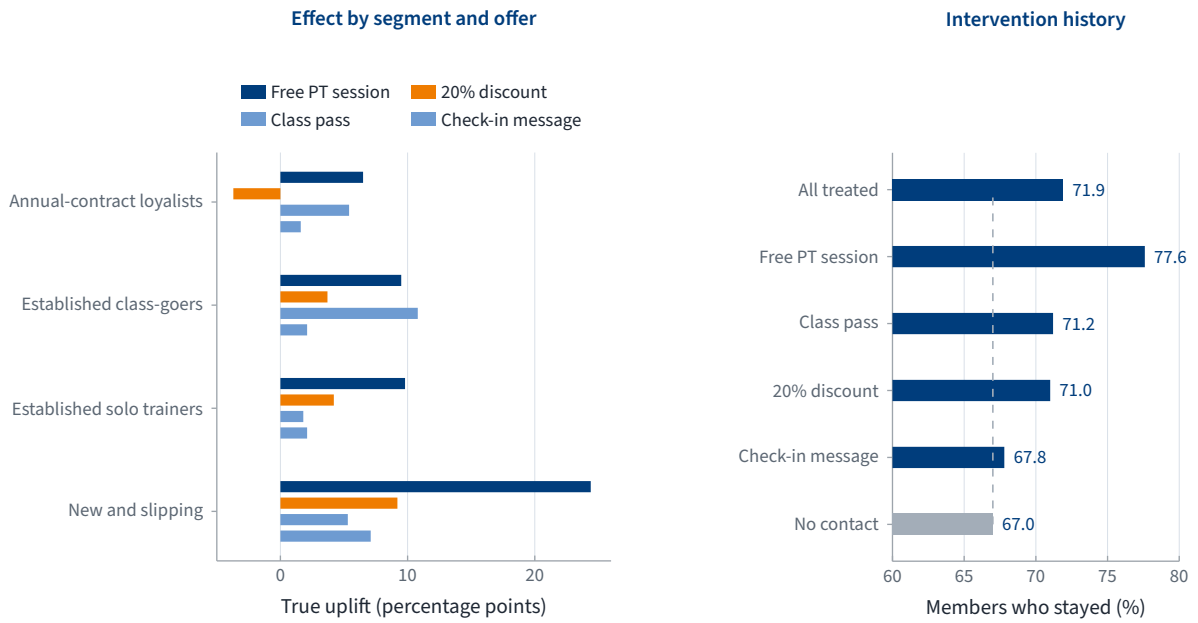


Figure 3.2 The scenario's ground truth. Left: mean true uplift by segment (Section 4.1) and offer; a discount backfires on annual members. Right: share of members who stayed in each arm of the 2,400-contact history, against the 984 controls (dashed line).

Source: `results/a5_true_uplift_by_segment.csv`; `a_scenario_uplift.ipynb`, cell 2.

Table 3.3 The retention offers.

Offer	Cost	Staff minutes	Weekly stock	Channels
Personal check-in message	$ 0	5	unlimited	SMS, email, call
Free personal training session	$40	60	40	call, in person
Bring-a-friend class pass	$15	10	100	SMS, email, call
One month at 20% off	$12	15	unlimited	email, call

Gym structure. Three clubs, each with three instructors who teach two classes. The 1,611 members who attend group classes are spread over one to three classes at their club, and their class visits add up to their real Kaggle visit frequency, so the synthetic timetable never contradicts the real data.

Offers. Four retention offers with a cost, staff time, weekly stock, allowed channels and the skill they need (Table 3.3). The same catalogue seeds the application, where an administrator can edit it.

Ground truth. Each member has a churn hazard for the next month without contact, computed from features only and never from the real label; as a plausibility check it correlates 0.715 with that label. Each offer has a documented, heterogeneous effect on the hazard: a training session helps new members most, a class pass helps class-goers, and a discount helps price-sensitive monthly members but backfires on annual ones. The resulting *true uplift*, the gain in the probability of staying, exists for all 16,000 member–offer pairs and averages +5.8 points (Figure 3.2, left).

Gym events. In the week before the scoring date, the instructor with the most attendees leaves (Maya Chen, Zumba and Core, club C2) and the busiest class at another club is cancelled (HIIT, club C1). Each event raises the hazard of the members it touches in proportion to the share of their class visits it hits. The churn model cannot see these events; only the knowledge graph can, which is what E7 tests.

Intervention history. 2,400 past contacts in the twelve weeks before the scoring date: 1,416 members received a random offer through a random allowed channel and 984 were left alone as controls, with outcomes drawn from the ground truth. Assignment is random, as in a trial, so an uplift estimator faces small samples and heterogeneity rather than confounding. The effects are small against the noise of yes-or-no outcomes (Figure 3.2, right), which is the realistic difficulty the case-based reasoner and the T-learner must overcome.

3.4 Data contract and reproducibility

Twelve tables form the hand-over from data to the rest of the system: the members and their features, the synthetic gym, the history, the events, the two ground-truth tables, and the model outputs (Chapter C). Their columns are fixed in code, and every build checks keys and value rules and fails rather than write a broken table. A small fixture with the same columns let the rules, planner and knowledge graph be built before the real tables existed.

One command, `anchorai data build`, regenerates every table, model and score from the raw file in about 20 seconds. Every random step takes one seed (42 for every reported result), the events follow from the attendance rather than from chance, and two builds with the same seed are identical. The build code was frozen on 17 October 2026, so every experiment in Chapter 9 ran on the same tables.

4 Member intelligence: segments, churn and uplift

In brief

- Every member receives a plain-English **segment**, a calibrated **churn probability** with a risk band and its five drivers, and an **uplift estimate** for each offer.
- Calibration is not cosmetic: the planner multiplies probability by uplift and value, so a distorted probability silently distorts the plan.
- The results are measured in E1–E3 (Chapter 9).

The three outputs are written by the data build and served by the API as one record per member, the contract between member intelligence and the rest of the system. Rules, planner, explanations, assistant and knowledge graph all read it.

4.1 Behavioural segmentation

Staff think in groups, such as 'new members who have stopped coming'. Segments give the dashboard, the member list and Member 360 that vocabulary. They describe behaviour only: they are not an input to the churn model, and no decision rule depends on them.

K-means [11] is fitted on the training split over seven standardised behavioural features (months as a member, log-scaled because it is long-tailed; lifetime weekly visits; visit trend; contract length; extra spend; age; group-class attendance). The churn label is never used; each segment's churn rate is computed only after the clusters are fixed. Every k from 2 to 10 is fitted with ten initialisations and scored by silhouette [12]; stability is the adjusted Rand index [13] against refits with other seeds. Because the silhouette is low and almost flat on this data, a fixed rule chooses k: the most detailed k between 3 and 8 whose silhouette is within 5% of the best. Segments are then ordered by churn and named by fixed rules on their centres (Table 4.1), so their names survive a rebuild with another seed or a new data file. The proposal's Gaussian mixture was dropped to keep one method.

Table 4.1 Segment naming rules, checked in order on each segment centre. With seed 42, four of the six names occur.

Rule on the segment centre	Name
under 2 months as a member, and visits this month below 0.8 × the usual rate	New and slipping
under 2 months, with steady or rising visits	New and settling in
average contract of 9 months or more	Annual-contract loyalists
under 1.3 visits a week	Established but infrequent
at least half attend group classes	Established class-goers
otherwise	Established solo trainers

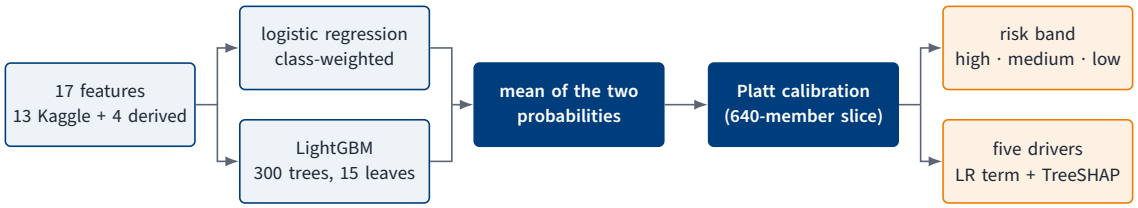


Figure 4.1 The churn pipeline. Both members of the ensemble are fitted on 2,560 training members; the remaining 640 calibrate the probability and set the decision threshold. Training members are scored out of fold.

4.2 Churn model

The model is the mean of two classifiers over the 17 features (Figure 4.1): a class-weighted logistic regression, stable and easy to read, and LightGBM [14], which captures the non-linear effects and interactions a linear model cannot. This is the ensemble the proposal named, with three choices that matter.

Imbalance. With 26.5% churners, SMOTE-NC oversampling [15], applied inside each training fold so no synthetic row reaches a validation fold, was compared with class weights in five-fold cross-validation. They scored almost the same, and class weights were kept on the pre-declared tie-break (PR-AUC, then Brier) and because they add no synthetic members.

Calibration. Both treatments distort probabilities: a model trained as if churners were common predicts churn too often. A Platt calibrator [16] is therefore fitted on a held-out 20% of the training split, never on the test set. A two-parameter fit suits a slice this small, where isotonic regression tends to overfit [17]. The decision threshold maximises F1 on the same slice, but staff see bands rather than a threshold: high from 0.6, medium from 0.3, low below. A band is always shown as text and colour, never colour alone, and a probability is never shown as 0% or 100%.

Honest scores for every member. Test members are scored by the final model. Each training member is scored *out of fold*, by a model fitted without them, so no displayed score has seen its own label; a regression test checks that changing one member's label never changes that member's score.

Drivers. The five largest contributions to a member's log-odds become the 'Why this risk score' chart on Member 360 and the drivers in every plan explanation. Each contribution averages the logistic regression's term (coefficient times standardised deviation) with LightGBM's exact TreeSHAP value [18, 19]. Figure 4.2 shows the global picture: falling visits and a short membership push hardest towards churn. The interface states that drivers explain the model's estimate and are not proof of cause.

4.3 Uplift baseline

A churn score says who is likely to leave, not whom a contact would keep, and on field data targeting by expected response reduces churn more than targeting by risk [4]. The planner therefore spends on *uplift*: the change in the probability of staying that an offer causes. Uplift modelling is not taught in the course; the T-learner [20], the simplest meta-learner, was self-researched as the benchmark for the case-based estimator (Section 6.2). It fits one outcome model per arm on the randomised history, one per offer (341 to 368 treated members each) and one for the 984 controls, each the mean of a logistic regression and a small LightGBM; a member's uplift for an offer is the predicted probability of staying

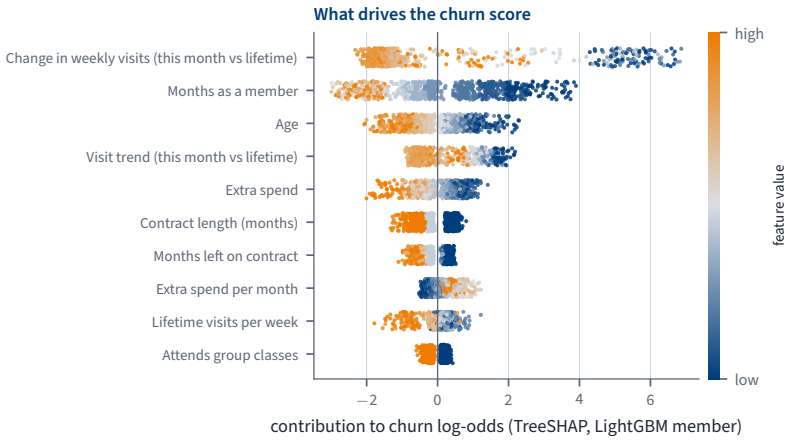


Figure 4.2 TreeSHAP values of the LightGBM member on the 800 test members, for the ten most influential features. Each dot is a member, coloured by its feature value; dots to the right push towards churn. Real Kaggle data.

Source: `scripts/figures.py` (`churn`), as in `a_churn.ipynb`, cell 5.

under that offer minus that under no contact. Its recovery of the true uplift, and its comparison with case-based reasoning, are reported in E3 (Section 9.3).

5 Decision automation: rules and knowledge acquisition

In brief

- The churn model says who is at risk; the rule base says what staff may do about it. For every member it returns the allowed offers, a channel for each, and the reason.
- 29 rules in readable YAML, each citing its source, run on a forward-chaining engine whose guardrails always win.
- Rules are versioned, validated before saving and testable on any member from the application.

5.1 The rule engine

The engine is a forward-chaining production system [21] (Figure 5.1). It matches rules against a working memory of facts, fires one, and repeats until nothing new matches. The agenda holds every enabled rule that has not fired and whose conditions all hold; a condition such as `trend_ratio <= 0.7` is parsed with a regular expression, never evaluated as code. Conflicts are resolved by salience, then specificity (more conditions first), then file order. A rule does exactly one thing: a *derive* rule asserts a fact, a *playbook* proposes an offer on a channel, and a *guardrail* blocks named offers or all of them. Each rule fires at most once, so the loop always ends; blocks are applied after the loop, so guardrails win because they are applied last, not because they fire first.

Three details keep the engine correct. A missing fact never matches, so a member who was never contacted simply does not trigger a rule on days since last contact; a fact of the wrong type never matches either. The first assertion of a fact wins, so a fact never changes after other rules have read it. And the output (facts, firings, proposals, blocked offers with the guardrail that blocked each, and allowed offers) is the contract with the planner.

5.2 The rule base

The 29 rules sit in three YAML files that an administrator can read and change. Eight *derive* rules name a pattern once so that playbooks and traces read in plain words: falling regular, new member, disengaged (two routes), high value at risk, social member, and routine disrupted when an anchor instructor leaves or a class is cancelled. Sixteen *playbooks* and five *guardrails* follow in Table 5.1. Every rule must name its source: the simulated interviews, a team assumption or policy, offer-catalogue eligibility, or a review finding. The rules read a declared vocabulary of facts with types: the model's

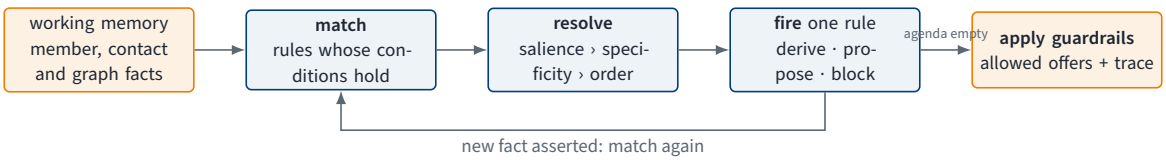


Figure 5.1 The recognise–act cycle. Each rule fires at most once (refraction); blocked offers are removed after the loop, so a guardrail always overrides a playbook.

Table 5.1 Playbooks and guardrails. Each playbook has a twin for members with no phone on file, which uses email or an in-person contact instead of SMS or a call.

Rule	When	Proposes or blocks	Channel	Salience
P03	routine disrupted (anchor instructor left, class cancelled)	check-in message	SMS	45
P01	new and slipping	free PT session	call	40
P02	falling regular	check-in message	call	40
P06	high-value member at risk	free PT session	call	35
P04	joined via a friend, medium or high risk	class pass	SMS	30
P05	month-to-month, high risk	20% discount	call	25
P07	disengaged, medium or high risk	check-in message	SMS	20
P08	medium risk	check-in message	SMS	10
G01	contacts in 30 days at the cap (default 1)	blocks all offers		
G02	offers in 90 days at the cap (default 2)	blocks all offers		
G03	contract longer than one month	blocks the discount		
G04	low risk band	blocks paid offers		
G05	free PT session in the last 90 days	blocks free PT		

Source: `reasoning/rules/playbook.yaml`, `guardrails.yaml`.

features and scores; contact facts that combine the synthetic history with every outcome staff log in the application, so next week's plan cannot re-contact this week's members; the two caps set on the Offers page, so an administrator tightens a guardrail without editing a rule; and knowledge-graph facts, which are simply absent when the graph is down, leaving the remaining rules to run.

Validation. Before a rule base is used, a validator rejects duplicate identifiers, unknown facts, offers or channels, literals of the wrong type, contradictory conditions on one fact, derive rules that overwrite a measured fact and channels an offer does not support; it warns about playbooks a guardrail always blocks, conditions no rule can make true and inactive offers. The shipped rule base has no issues, and 29 hand-built regression members state which rules must fire, which must stay silent and which offers must be allowed or blocked (E6).

5.3 Knowledge acquisition

Rules were drafted from data first. A notebook runs each draft rule over all 4,000 members, showing churn by bin on the training split only, how often each rule fires, and whether each derived pattern separates churners from the base rate; thresholds were never tuned on the scenario's true uplift, on which E4 and E6 are scored.

The planned practitioner interviews could not take place. Instead, three interviews were **simulated**: an AI model played an independent owner-manager, an area manager and a membership lead from written briefs, answering the interview script without seeing the rule base. We treat the answers as plausible expert opinion, not evidence; each transcript and each rule they influenced says so. They changed the rule base in six ways: the falling-regular threshold became a compromise between their view and the data (trend at most 0.7); 'disengaged' now also requires a falling trend, so habitually infrequent members are not flagged; a new rule catches new members with almost no visits; a free session for a fading regular became a check-in message, which felt less like selling; discounts were restricted to month-to-month members and moved last; and SMS replaced email for low-touch contacts.

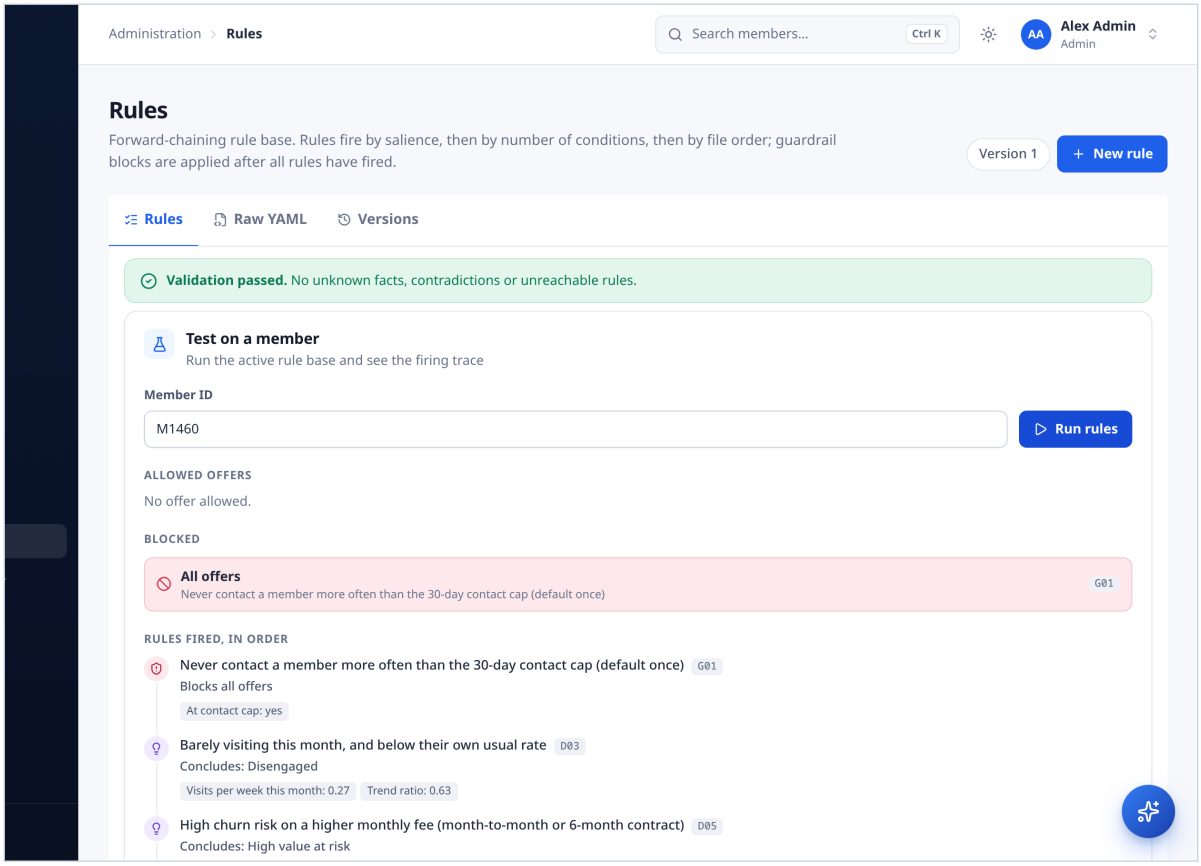


Figure 5.2 Testing the rule base on member M1460 in the Rules page. A contact in the last 30 days triggers guardrail G01, which blocks all offers; the rules that fired follow in order, each with the facts it matched.

The stricter team defaults for the contact caps were kept, a contract-ending rule was dropped for lack of signal, and consent is left to the gym's messaging tool, as no opt-out fact exists.

5.4 Administration and explainability

Administrators manage the rules in the application (Figure 5.2). Every save creates a numbered version with a mandatory note and author; nothing is overwritten. A draft is parsed and validated before it can be saved, can be tested on any member, and can be rolled back to any earlier version, with every write audited. Concurrent edits are detected: a save that names an outdated version is refused. When a manager adds or changes a contact in a plan, a live check reports whether the offer is allowed, which guardrail it breaks and whether a playbook proposed it, so an override is always an informed one.

The firing trace is the rule part of every explanation. Each firing records the rule, its description, the facts it matched and its effect; the trace travels with every plan item and appears on Member 360, in the plan's 'why' panel and in the assistant's answers, with derived facts, proposals and blocks colour-coded.

6 Decision and optimisation: CBR, GA and CP-SAT

In brief

- Case-based reasoning estimates what each allowed offer would achieve for each member, and shows the precedents behind every estimate.
- A genetic algorithm chooses the most valuable plan within the week's limits, including effects between members who train together; CP-SAT gives an exact benchmark and an exact planner for the linear case.
- A policy evaluator grades every plan against the scenario's true effects and is never used to choose one.

6.1 The planning problem

Each allowed member–offer–channel triple from the rules is an *option*; in the experiments 1,045 members had 2,025 options, at most four each, and not contacting a member is always possible. An option's value is the expected revenue it keeps, net of its cost,

$$v_{mo} = \hat{u}_{mo}\, f_m\, H - c_o, \qquad H = 6 \text{ months}, \tag{6.1}$$

where $\hat{u}_{mo}$ is the estimated uplift of offer o for member m [22], f_m the monthly fee and c_o the offer's cost; the six-month horizon is deliberately cautious. A plan is feasible if it contacts each member at most once and stays within the budget, the staff minutes, the trainers' minutes for training sessions and every offer's stock. Contact caps and eligibility are already enforced by the rules.

The *linear* objective, the sum of chosen values, makes this a multiple-choice knapsack, which an exact solver handles quickly. The *full* objective adds two effects between members using the class groups from the knowledge graph. *Spillover*: members who train together influence each other, so a contacted member's value is multiplied by $(1 + \beta s_m)$, where s_m is the share of their classmates also contacted and $\beta = 0.5$. *Diminishing returns*: contacts aimed at one instructor's classes overlap, so the n-th most valuable of them counts with weight $1/(1 + d(n-1))$, $d = 0.02$. These effects make the problem non-linear and justify a search method. They are planning assumptions, absent from the scenario's ground truth, so E5 tests the genetic algorithm across a range of β and d.

6.2 Case-based reasoning

A case is one past contact decision: a member, the offer received or none, and whether the member stayed. The case base starts with the 2,400 randomised cases of the scenario's history. AnchorAI follows the classical retrieve–reuse–revise–retain cycle [23] (Figure 6.1).

Retrieve finds, for a member and an offer, the 200 nearest cases among those who received that offer or nothing, in one shared neighbourhood; a member never retrieves their own case. *Reuse* treats the neighbourhood as a small A/B test: the local uplift is the stay rate of neighbours who received the offer minus that of neighbours who did not, closer cases weighted more. *Revise* shrinks this noisy local estimate towards the offer's average effect, with a weight equal to the estimated real variation between members over that variation plus the noise; where members barely differ, the estimate falls back to

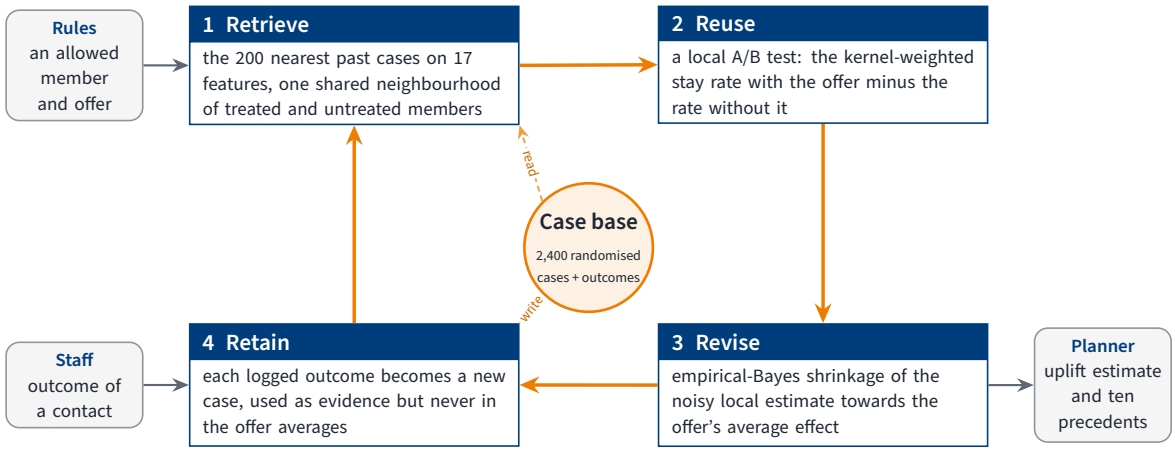


Figure 6.1 The case-based reasoner. The same neighbours that produce an estimate are shown to staff as its precedents: the ten most similar members who received the offer or nothing, each with what happened to them.

Table 6.1 Design of the genetic algorithm (DEAP [8], fixed seed).

Element	Choice
Chromosome	one gene per member with an option; its value picks the option, 0 means no contact
Fitness	the full objective, with spillover and diminishing returns
Population	100 plans: the greedy plan, 49 mutated copies of it and 50 sparse random plans
Operators	tournament selection of 3; uniform crossover, probability 0.7; about two genes mutated per child; 2 elite plans
Constraints	a **repair operator** drops the options with least value per share of budget and staff time until every limit holds
Stopping	300 generations, or 60 without improvement

the average. *Retain* adds each logged outcome as a new case (accepted as stayed, declined as left), a stand-in until the real outcome is known weeks later.

Three design decisions came from testing. A first version took treated and untreated neighbours separately and scored much better against the truth, but a *null test* with shuffled offers, whose true effect is zero, showed it was measuring churn risk rather than the offer: the two groups were not equally similar. The shared neighbourhood passes the test. Second, on the history alone, every weighting, every k and every variant scored within the noise of a random ranking by the Qini coefficient [22], so the settings (equal weights, $k = 200$, distance weighting) were fixed by rule before grading against the truth. Third, logged outcomes never change the offer averages: the planner chooses whom to contact, so its own outcomes are not a random sample and would bias them. Learning reliable averages from live use would need a small random control group each week.

6.3 Genetic algorithm

Table 6.1 lists the design. Repair was preferred to penalties, which let infeasible plans breed and waste generations on them, and seeding with the greedy plan gives the search a strong start; E5 confirms both choices. The algorithm is the default planner and takes about 25 seconds for the gym's 1,045 candidate members, run as a background job with a progress bar.

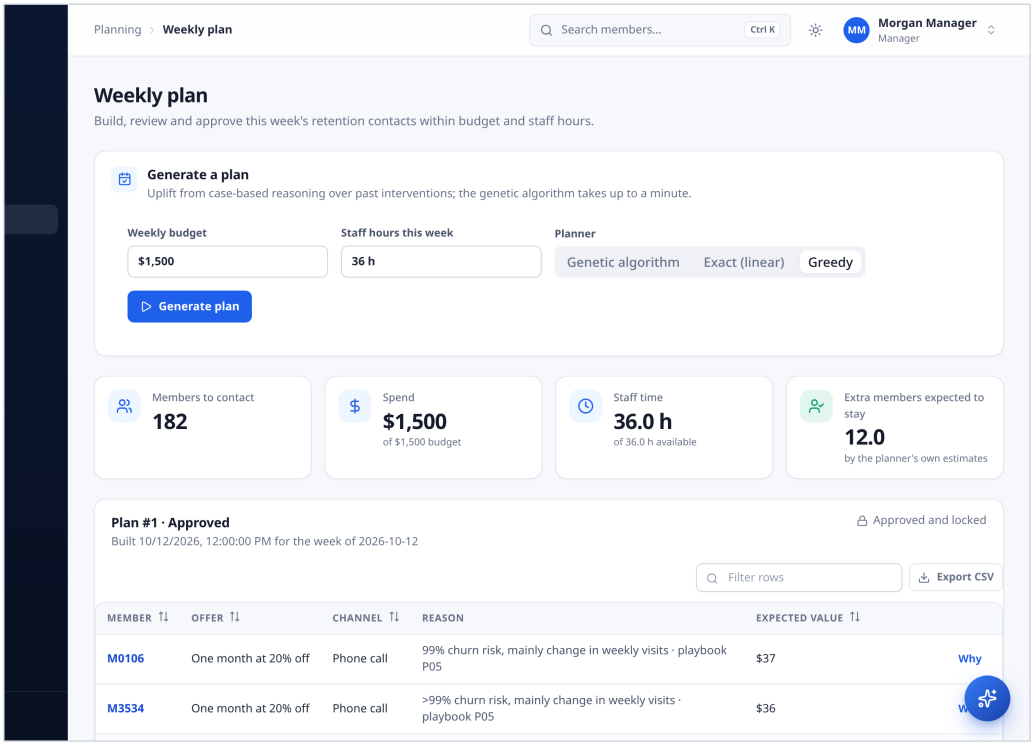


Figure 6.2 The weekly plan for the seeded demo week: budget, staff hours and planner, then the approved plan's 182 contacts, each with its offer, channel, reason, expected value and a 'why' link to the full explanation.

6.4 CP-SAT exact planner and benchmark

CP-SAT [9] takes a declarative model and returns either a proven optimum or its best plan with an upper bound. The *linear* model (one binary variable per option, at most one per member, and the budget, staff, trainer and stock constraints) proves the optimum for the whole gym in about a second; managers can select it as the exact planner, and the assistant uses it to answer what-if questions about the budget. The *full* model adds spillover as products of binary variables and diminishing returns as rank slots per instructor group, and is exact rather than approximate. It proves optimality for instances of up to about 50 members and measures how close the genetic algorithm comes (E5).

6.5 Baselines, evaluator and the path to action

Three simple policies show what the planner adds. All fill the plan by taking options in order while they fit every limit, and differ only in the order: *contact-all* takes each member's top playbook offer in random order, as a calendar campaign would; *risk-top-k* takes it by churn risk, as a vendor score would; *greedy-uplift* takes any option worth more than it costs, by value per share of budget and staff time. The *policy evaluator* grades any plan against the scenario's true effects; it is used only to grade, so no planner can learn the answer.

A plan becomes a working list through the manager's review (Figure 6.2): each item carries its explanation; overrides need a reason and pass an immediate rule check; the rest of the week can be re-optimised around them; and approval locks the plan, one per week. Assignment and outcome logging follow as described in Figure 2.2.

7 Knowledge graph and staff assistant

In brief

- A Neo4j knowledge graph links members to the classes, instructors and clubs they use, finds who a gym event hurts, and turns that into rule facts and optimiser cohorts.
- A staff assistant answers questions in plain English through a fixed pipeline in which the language model only phrases evidence that deterministic tools have already computed.
- Both keep working, degraded, when Neo4j or the language model is down.

7.1 Knowledge graph

The graph [24] is built from the data tables with idempotent, batched `MERGE` statements (Figure 7.1). It reads the *effective* roster, the generated instructors, classes and events with administrators' edits applied on top, so the graph reflects who teaches what today and a data rebuild never loses an edit. Every query is a fixed Cypher template that receives values only as parameters and runs in a read-only transaction, which Neo4j itself enforces.

Signals as rule facts. Three templates turn the graph into facts per member: the share of their class visits taught by instructors who have left (flagged as a lost anchor instructor at 0.5 or more), whether a class they attend was cancelled, and the mean churn probability of their class peers. The facts are cached in a file that the rule engine reads, so the rules fire the same when Neo4j is stopped. The peer-risk fact is kept but not shown to staff: on the real data it varies only between 0.15 and 0.21 and is uncorrelated with a member's own risk. Against the Kaggle label the event flags show no lift, as expected, since the label knows nothing of the synthetic events; E7 measures them against the scenario's event effect instead.

Cohorts, simulation and freshness. The graph gives the genetic algorithm one row per member and class, weighted by the member's share of visits and attributed to the instructor who teaches it now. A what-if simulation answers 'what if this instructor left, or this class were cancelled?' without changing anything: it adds the exposure to the affected members' in-memory facts, reruns the rule engine and reports who is affected, how many are high risk and which rules newly fire (Figure 7.2). A reload takes about a second; while it runs, reads fail fast rather than see a half-loaded graph, and the graph is marked stale, with the reason shown on dependent pages, whenever the cached facts are older than the last data build or a roster edit.

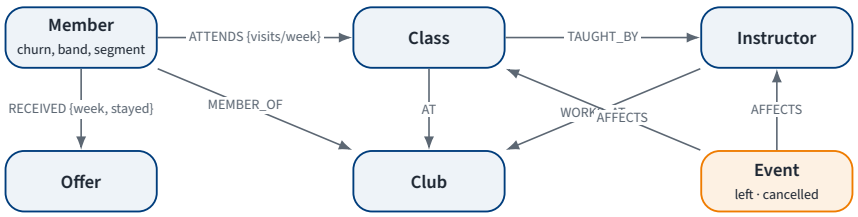


Figure 7.1 The knowledge-graph schema. On the demo data: 4,000 members, 3 clubs, 9 instructors, 18 classes, 2,890 attendance links, 4 offers, 1,416 received offers and 2 events.

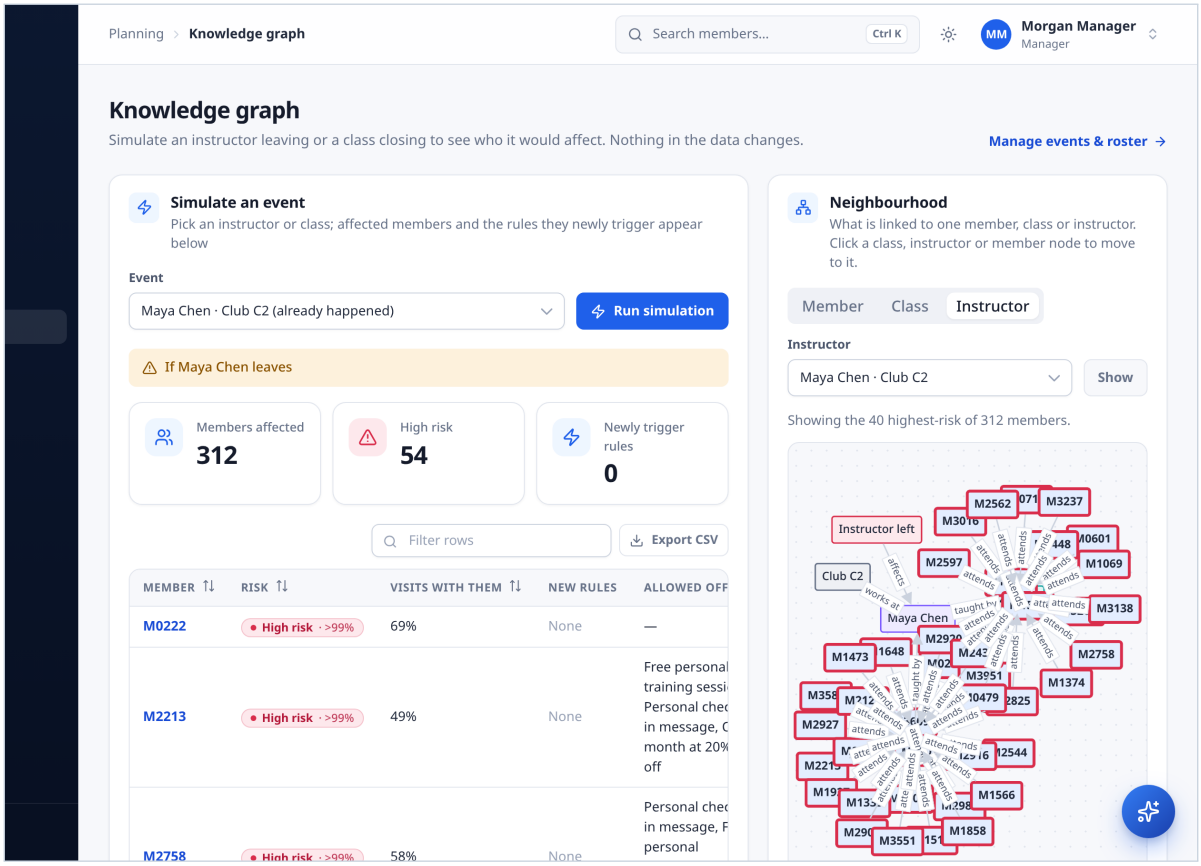


Figure 7.2 Simulating the departure of instructor Maya Chen in the graph explorer: 312 members are affected, 54 of them at high risk. Right: her neighbourhood, with the 40 highest-risk of her 312 members drawn.

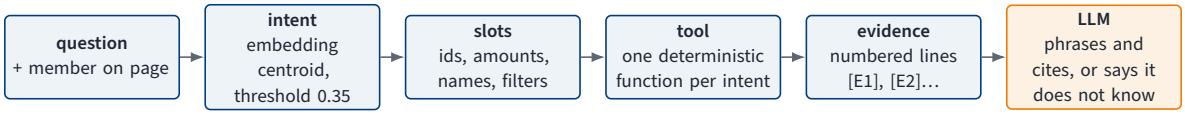


Figure 7.3 The assistant pipeline. The language model never chooses a tool and never sees data a tool did not return; without it, the evidence lines are returned as a cited list.

7.2 Staff assistant

The assistant is a pipeline with the language model at the end (Figure 7.3), in the spirit of retrieval-augmented generation [25] but with deterministic retrieval. Seven intents cover what staff ask: why a member is at risk, what to offer them and why, why one member ranks above another, who an event affects, a ranked list of members, what is in the plan, and how the plan would change with another budget or staff hours.

Intent and slots. Each intent's example questions are embedded with a sentence-transformer [26] and averaged into a centroid; a question goes to the nearest centroid, or to 'unknown' below a similarity of 0.35. Two rules sit on top: an intent that needs member identifiers cannot win without them (the member open on the page fills in), and questions that ask to change data are refused. Slots are filled by patterns and roster look-ups in a fixed order, money before member identifiers, so 'budget drops to 1000' never becomes member M1000. The design was chosen on AI-written questions, which are optimistic; E8 tests it on questions written by teammates who never saw the training examples.

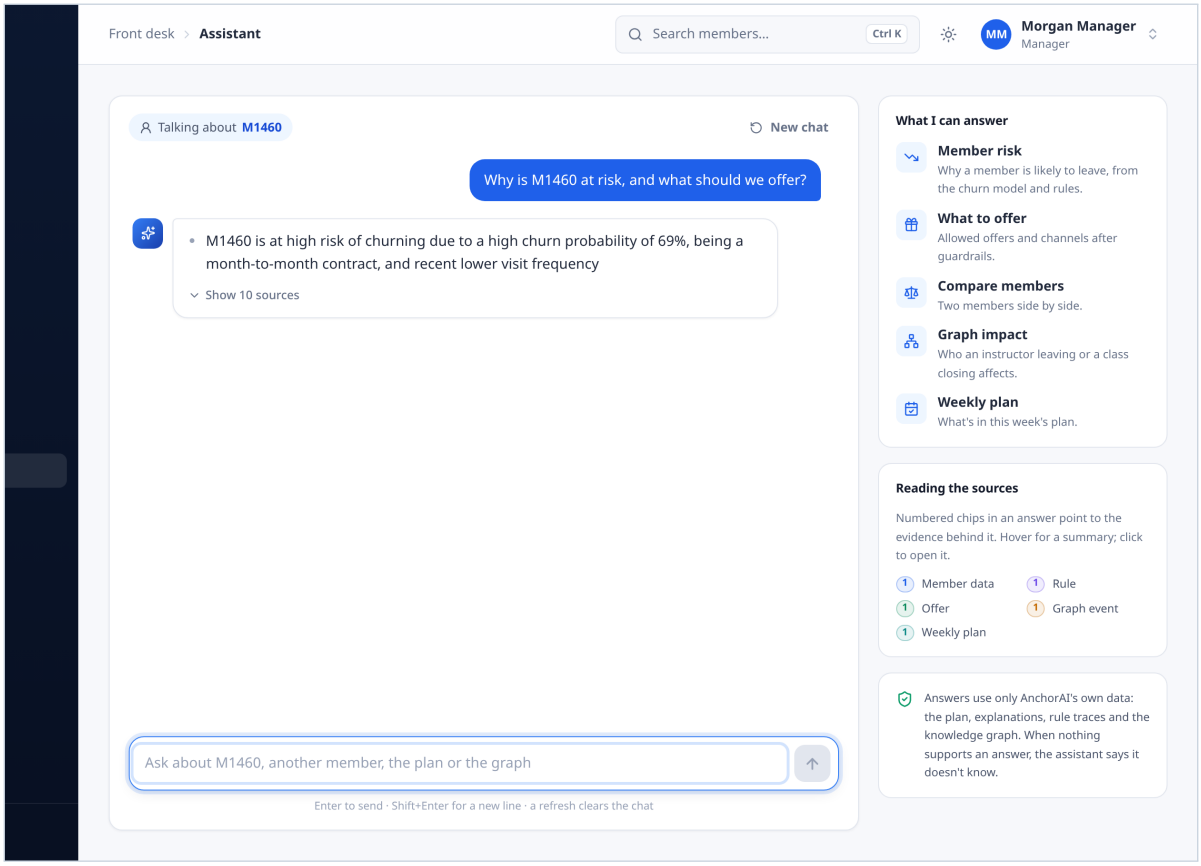


Figure 7.4 The assistant answering a manager about member M1460 with the local `qwen2.5:7b-instruct` model. The answer cites its sources, which open as evidence chips linking to the member, rule or case.

Tools and grounding. Each intent has one tool that returns numbered evidence lines from the services the API already uses. The what-if tool runs the same exact planner as the saved plan, twice, and reports the difference without saving anything. The prompt allows the model to answer only from the latest evidence lines, to end every sentence with the identifiers it relies on, to invent no number, name, offer or reason, and otherwise to reply exactly ‘I don’t know based on the data I have’. The provider is set by configuration alone, a hosted API or a local Ollama model. Plan details are reserved for managers, both in the tools and in the evidence given to staff, and the browser keeps the conversation, so the server stores none.

Message drafts. For an allowed offer and channel, the assistant drafts a message that staff edit and send from their own tool; nothing is sent automatically. The request passes the same rule check as the plan, a template built only from evidence always works, and a language-model rewrite is rejected in favour of the template if it mentions churn or risk, contains a placeholder or a number not in the evidence, or is too long for an SMS.

8 System development and implementation

In brief

- One command takes a fresh clone to a running, seeded system; a rebuild from the application takes effect without a restart.
- Sixteen pages serve three roles, and Member 360 brings every component's reasoning about one member onto one screen.
- 195 offline tests guard against the errors that would quietly invalidate a result; the system degrades rather than fails when a service is down.

8.1 From raw file to served scores

`anchorai setup` builds the data tables and models from the raw Kaggle file in about 20 seconds, migrates and seeds the application database, including a demo week of approved plans and logged outcomes, and loads the knowledge graph. The API loads the build once and re-reads it whenever the model card changes, so a rebuild takes effect on the next request; before anything is built, every member endpoint returns a clear 'not built yet' message that the pages display. An administrator can upload a new file and rebuild with any seed from the Data & models page: the upload is validated (size, all 14 columns, at least 100 rows), the rebuild runs in the background while the old build keeps serving, a second rebuild is refused while one runs, and the knowledge graph is then reloaded or marked stale if Neo4j is down. Every write, here and elsewhere, is audited with the user, time, and values before and after.

Table 8.1 Pages by role. Every page has loading, empty and error states, and roles are enforced again by the API.

Page	Roles	Purpose
Dashboard, Segments, Members	manager, admin	risk at a glance; segment cards with profiles; a filterable, exportable member list
Member 360	all (staff read-only)	everything the system knows and recommends about one member
Weekly plan, Scenarios	manager, admin	build, review, override and approve the plan; compare budgets and staff hours side by side
Knowledge graph, Events & roster	manager, admin	explore neighbourhoods and simulate events; record departures, cancellations and reassignments
My contacts, Assistant	all	a call sheet with talking points, drafts and two-tap outcome logging; the staff assistant
Rules, Offers, Data & models	admin	versioned rule editing and testing; offers, roster and planning constraints; model card and rebuilds
Users, Audit log, Settings	admin	accounts and roles; every change with before and after; service health

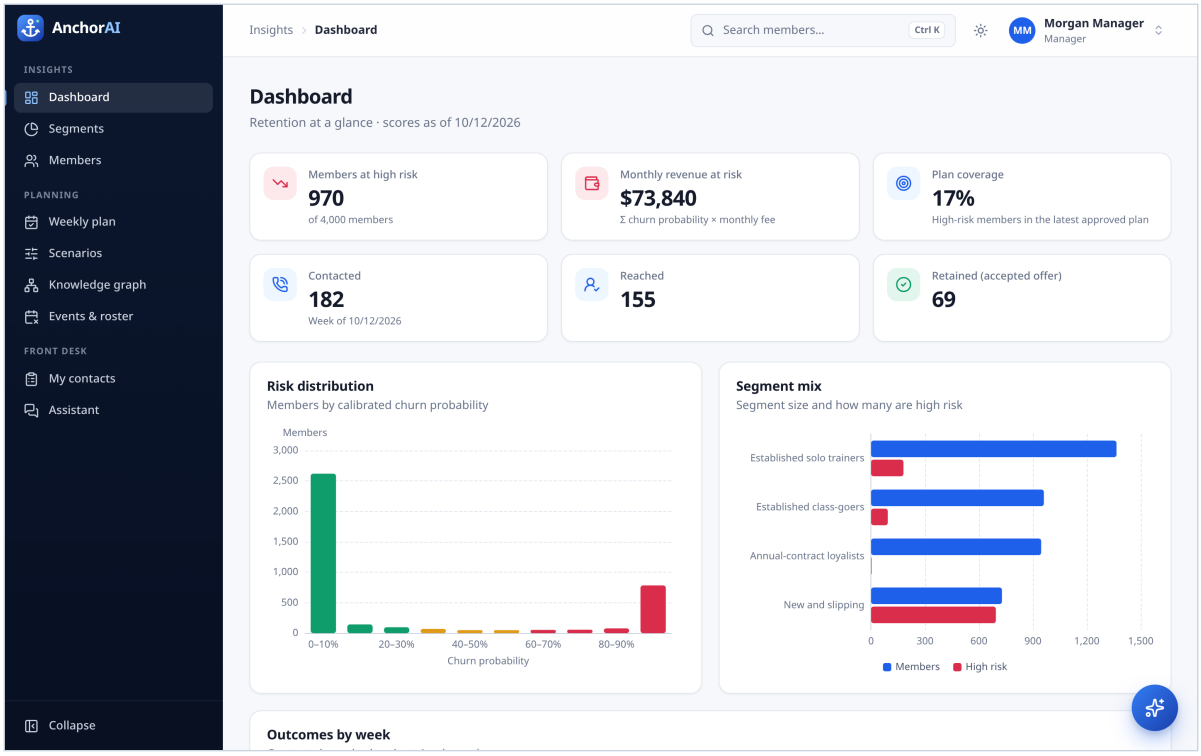


Figure 8.1 The manager's dashboard on the demo data: members at high risk, monthly revenue at risk (the sum of churn probability × monthly fee), plan coverage of high-risk members, contacts reached and retained in the latest week, the risk distribution and the segment mix. The sidebar groups pages by task: insights, planning and front desk.

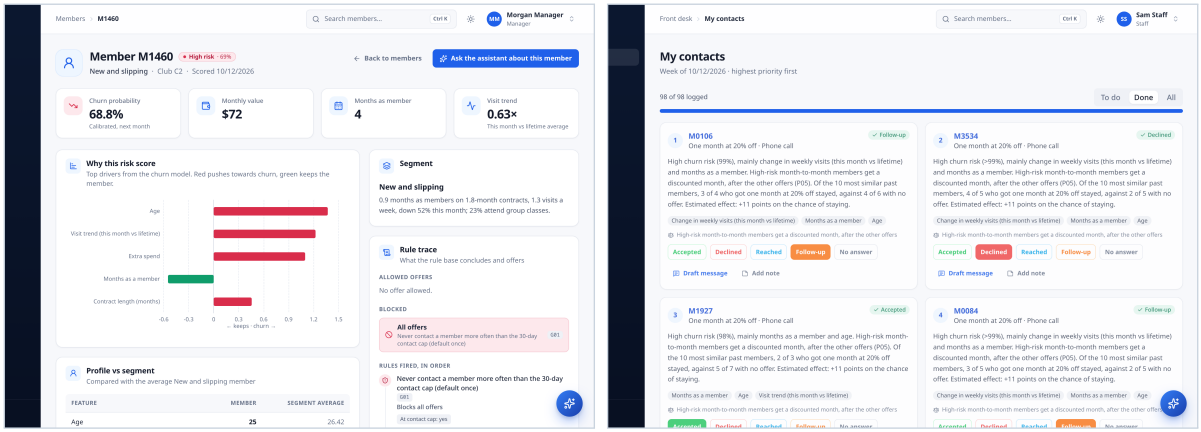
8.2 The application

Table 8.1 lists the pages. The manager starts on the dashboard (Figure 8.1); the plan and the rules appear in Figures 5.2 and 6.2, the graph and assistant in Figures 7.2 and 7.4. **Member 360** brings the whole system together (Figure 8.2): the member's risk and its drivers, the profile against the segment, classes attended and past contacts come from member intelligence; the rule trace, with a draft message for each allowed offer, and the graph neighbourhood from the knowledge components; and the similar past cases and the contact history from the planner. Each panel calls its own endpoint and has its own loading and error state, so if one service is down only that panel says so. Staff work from **My contacts**, their call sheet in priority order with the reason, the talking points from the explanation and two-tap outcome buttons.

Usability and accessibility. Risk is always shown as text and colour ('High risk · 69%'), never colour alone, and a probability is never printed as 0% or 100%. Every error state offers 'Try again'; a global member search opens from the keyboard; inputs are labelled and cards are focusable; tables export to CSV on the server, so an export includes every filtered row, not just the visible page. The application has light and dark themes, and staff pages work on a tablet.

8.3 Degraded operation

The planner needs neither Neo4j nor the language model (Table 8.2). A fallback answer is built from the same evidence the model would have phrased, so it remains grounded and cited.



(a) Member 360 for M1460: high risk (69%), the five drivers, the profile against the segment and the rule trace.

(b) A staff member's call sheet: each contact with its offer, channel, reason, precedents and logged outcome.

Figure 8.2 The two screens where a recommendation meets the people who act on it.

Table 8.2 Behaviour when a service is unavailable. Each row is covered by an automated test with the service mocked.

Unavailable	Still works	Lost
Neo4j	scores, rules (from cached graph facts), plans, optimiser cohorts, six of seven assistant intents, drafts	graph explorer and simulation; graph-impact answers; edits wait for the next reload
Language model	intents, slots and tools; answers become the cited evidence lines; drafts use the template	natural phrasing; rephrased follow-ups
Embedding model	the rest of the API	the assistant reports that its intent model is unavailable

8.4 Testing and quality assurance

The suite has 195 tests, which run offline in under a minute with Neo4j and the language model mocked. They target errors that would quietly invalidate a result rather than crash: test labels reaching the churn model (flipping every test label must leave every score unchanged) or a training member's score depending on its own label; a build that is not deterministic for a seed; tables drifting from the contract; events changing outcomes before their week; plans that break a limit; rule firing order and guardrail precedence; Cypher receiving values other than as parameters; the assistant turning money into member identifiers or its what-if disagreeing with the saved plan; and API roles, audit entries and status codes. Regression tests added during review were mutation-tested: each fails when the guard it protects is removed.

Three further checks complement them. The notebooks import from the package rather than copying its logic, and every number in this report was checked against their saved results. A fresh build from the downloaded raw file on 23 October reproduced the model card exactly, including the decision threshold of 0.404. And each part of the system was reviewed by a member who did not build it: one review of the authentication, database, rule engine, rule base and knowledge graph filed 32 findings (1 high, 8 medium, 23 low), each reproduced with a script or traced in the code, and all were fixed.

9 Experiments and findings

Eight experiments test the objectives of Section 1.5. E1 and E2 use only the 4,000 real members. E3–E7 are graded against the scenario layer's ground truth, which no estimator or planner ever reads, so they show that the system recovers effects of this kind, not how large real effects are. E8 evaluates the assistant on the system's own evidence. Every number below comes from a committed results file or notebook, all reproducible from seed 42.

9.1 E1: Are the segments meaningful?

REAL DATA Baseline: every k from 2 to 10

K-means was fitted on the 3,200 training members for each k and scored by silhouette, by stability across five other seeds (adjusted Rand index) and by how sharply the segments separate churn, which the clustering never sees.

Results. The documented rule chooses $k = 4$ (silhouette 0.174 against the best of 0.178 at $k = 3$); $k = 4$ adds a distinct annual-contract segment. Stability is very high: the mean adjusted Rand index across seeds is 1.00 (minimum 0.998). Test members fall into the segments in proportions within 4.2 percentage points of the training split, and churn separates by a factor of almost 70 (Figure 9.1).

Verdict ● Met. The segments are not geometrically crisp, as expected for behavioural data, but they are chosen by rule, stable, carry over to unseen members and separate risk in terms staff can name.

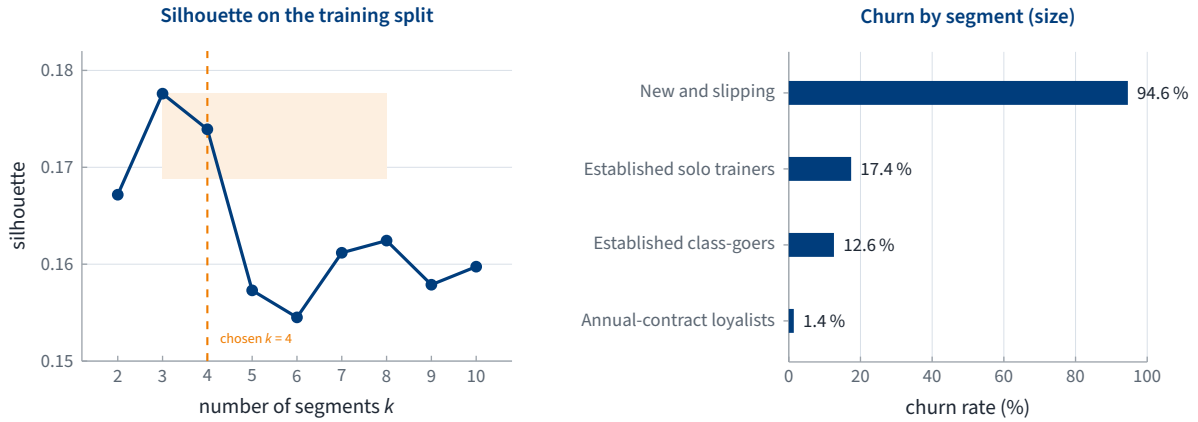


Figure 9.1 E1. Left: silhouette is low and nearly flat; the shaded band is within 5% of the best k between 3 and 8, and the rule picks the most detailed k inside it. Right: the four segments separate churn from 94.6% to 1.4% (sizes 728, 1,365, 961 and 946).
Source: `results/e1_silhouette_by_k.csv`, `results/e1_segment_profiles.csv`; `a_segmentation.ipynb`.

Table 9.1 E2. Test-set performance. The visit rule flags members with at most 1.63 visits a week this month, a threshold chosen on the training split.

Model	F1	Precision	Recall	PR-AUC	ROC-AUC	Brier
AnchorAI ensemble, calibrated	**0.908**	**0.906**	**0.910**	**0.973**	**0.988**	**0.035**
Single logistic regression	0.862	0.818	0.910	0.953	0.978	0.056
Visit rule	0.549	0.426	0.774	0.390	0.699	0.336
Ensemble without the five suspect features	0.817			0.900		0.069

Source: `results/e2_model_vs_baselines.csv`, `results/a_leakage.csv`.

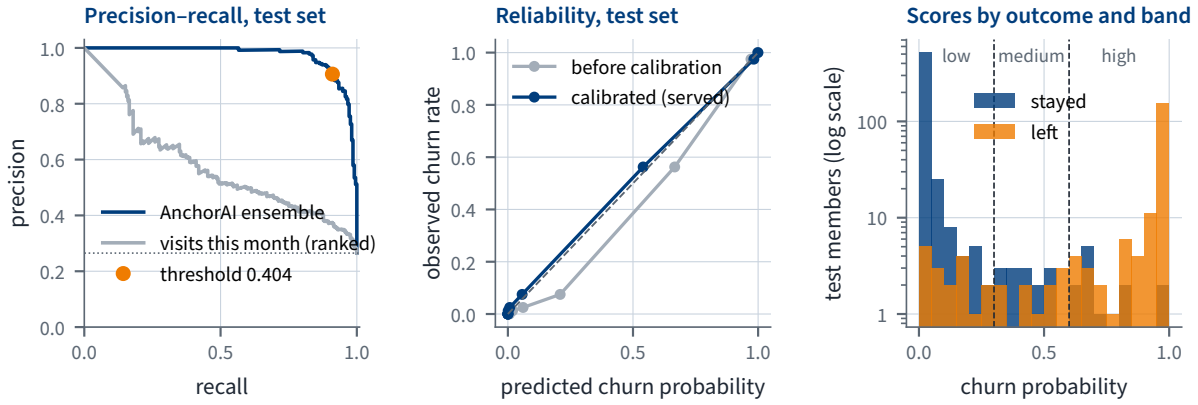


Figure 9.2 E2 on the test set. Left: precision–recall of the ensemble against members ranked by this month's visits, with the operating point at the threshold 0.404. Centre: reliability before and after Platt calibration. Right: scores by actual outcome, with the band cut-offs at 0.3 and 0.6.

Source: `scripts/figures.py` (`churn`), as in `a_churn.ipynb`, cells 3–4.

9.2 E2: Does the churn model beat simple rules?

REAL DATA Baselines: a visit-threshold rule; a single logistic regression

All metrics are on the 800 untouched test members, with churn as the positive class. A model that beats the baselines should do so on all three of F1 (the yes-or-no flag), PR-AUC (the ranking, suited to an imbalanced label [27]) and the Brier score [28] (the probabilities the planner consumes).

Results. The ensemble beats both baselines on every metric (Table 9.1). At the threshold of 0.404 it flags 193 of the 212 test churners with 20 false alarms: about one flag in eleven is a member who stayed. Class weights and SMOTE-NC are equivalent in cross-validation (PR-AUC 0.972 against 0.970, Brier 0.039 for both). Calibration moves the curve onto the diagonal (Figure 9.2), so each band churns at about its predicted rate: high-band members churn at 93.9% against 95.1% predicted, medium at 41.7% against 44.1%, low at 2.9% against 1.6%. Scores are extreme because a few columns nearly decide the outcome, not because of overfitting: training members, scored out of fold, err at 4.9% at 0.5 against 4.6% on the test set. Without the suspect same-month columns, F1 falls to 0.817, the honest lower bound if those columns were recorded after the churn month.

Fairness. Recall is 0.985, 0.943 and 0.522 for ages 18–25, 26–30 and 31+, and 0.861 and 0.955 by gender. The model misses about half of the rare churners aged 31 and over (23 in the test set); we report this rather than hide it, and suggest staff review medium-risk members of that group. The youngest group shows the opposite trade-off, with a false-positive rate of 14%.

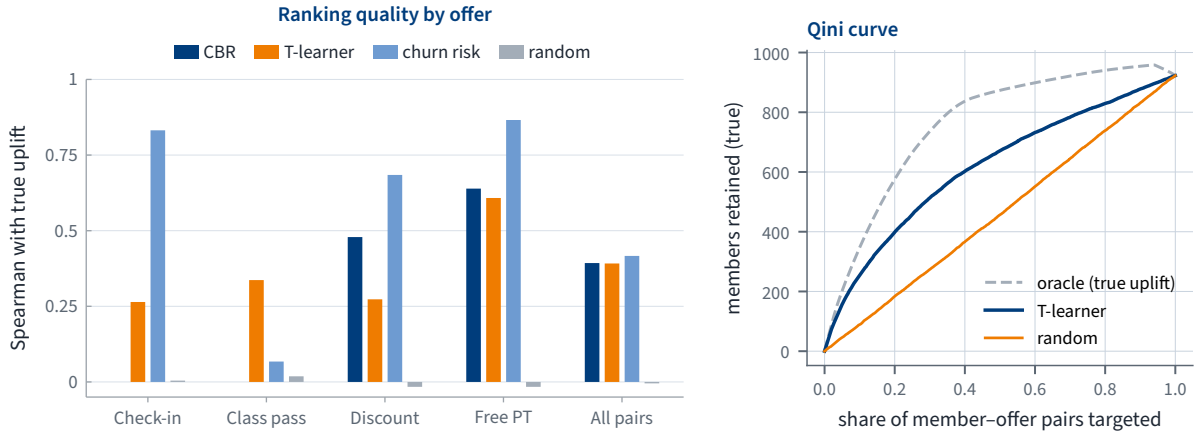


Figure 9.3 E3. Left: rank correlation between estimated and true uplift; CBR gives every member the offer's average for the check-in and the class pass, so it has no ranking there. Right: Qini curve of the T-learner against the oracle and random targeting.

Source: `results/e3_uplift_vs_truth.csv`, `results/a6_tlearner_recovery.csv`; `b_cbr.ipynb`, `a_scenario_uplift.ipynb`.

Table 9.2 E3. Extra members kept by greedy plans built on each estimator, graded by the true uplift.

Staff hours and budget	CBR	T-learner
36 h, $1,500 (default week)	21.0	**24.6**
72 h, $1,500	**36.6**	35.1
120 h, $4,000	**51.9**	50.7

Source: `results/e3_plan_quality.csv`.

Verdict ● Met. F1 0.908, PR-AUC 0.973 and Brier 0.035 against 0.862, 0.953 and 0.056 for logistic regression and 0.549, 0.390 and 0.336 for the visit rule, with calibrated probabilities the planner can use at face value.

9.3 E3: Can we find who is persuadable?

SCENARIO LAYER Baselines: T-learner; churn risk used as uplift; random

Estimates for all 16,000 member–offer pairs were compared with the true uplift by rank correlation and by the mean true uplift of each method's top 10% of pairs; the same estimates then drove the greedy planner, graded by the evaluator.

Results. CBR ties the T-learner overall (Spearman 0.39 each), is better for the two large-effect offers (free PT 0.64 against 0.61; discount 0.48 against 0.27) and picks the pairs with the highest true effect (top-decile mean 0.177 against 0.159). It cannot separate members for the check-in and the class pass, whose effects of a few points are smaller than the noise in about 350 randomised cases per offer, so shrinkage honestly returns the average. Churn risk ranks well *within* an offer, because scenario effects grow with risk, but says nothing about *which* offer helps, so its top picks are weaker (0.125). Plans built on the estimates (Table 9.2) favour the T-learner at the default 36 staff hours, where cheap check-ins dominate and CBR cannot rank them, and CBR with more staff time. A null test with shuffled offers confirms that CBR does not invent effects, and on the history alone every estimator, the T-learner included, ranks within the noise of random.

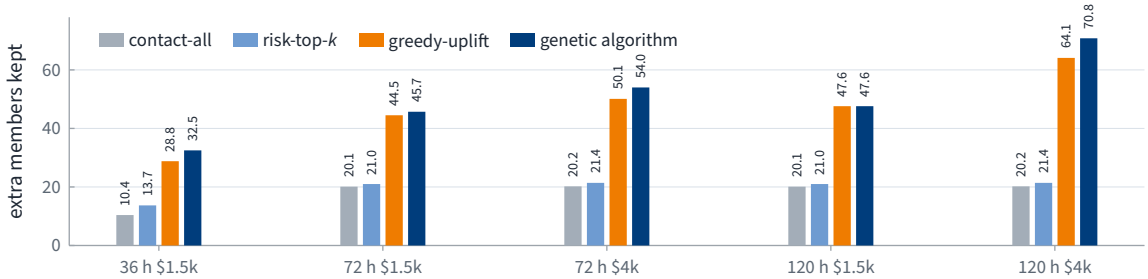


Figure 9.4 E4 (a), perfect knowledge: extra members kept by each planner, by staff hours and weekly budget. Planning by effect keeps 1.9 to 3.5 times as many members as contacting by schedule or by risk.
Source: `results/e4_optimiser.csv`, mean over seeds; `b_ga.ipynb`.

Table 9.3 E4 (b), estimated effects: extra members kept, as the system runs. Default configuration: the genetic algorithm on CBR estimates.

	Contact-	Risk-	CBR		T-learner	
Staff hours, budget	all	top-k	greedy	GA	greedy	GA
36 h, $1,500	10.4	13.7	21.0	16.7	24.6	**24.9**
72 h, $1,500	20.1	21.0	**36.6**	35.5	35.1	35.1
120 h, $4,000	20.2	21.4	**51.9**	48.1	50.7	50.7

Source: `results/e4_estimates.csv`, mean over seeds.

Verdict ● Partly met. CBR matches the T-learner overall, is better where effects are large and explains each estimate with real precedents; it is weaker where effects are small and staff time is scarce, which costs 3.7 retained members a week at the default settings. The team kept CBR as the planner's estimator for its inspectable precedents, with the T-learner as its benchmark.

9.4 E4: Does AnchorAI retain more members per dollar?

SCENARIO LAYER Baselines: contact-all; risk-top-k; greedy-uplift

Every planner ran at 36, 72 and 120 staff hours and several budgets, the genetic algorithm with three seeds, and every plan was graded by the evaluator as extra members kept beyond the roughly 2,660 of 4,000 who would stay anyway. Variant (a) gives every planner the true effects, isolating search quality; variant (b) gives them the estimates, as in real use. None of the 198 plans broke a limit.

Results. With perfect knowledge the genetic algorithm beats greedy wherever two limits bind together and ties where one dominates (Figure 9.4). On estimates (Table 9.3), every effect-based plan beats contacting by schedule or risk, and per staff hour at the default week the default configuration keeps 0.46 members against 0.29 for contact-all and 0.38 for risk-top-k. But the genetic algorithm's advantage reverses on noisy estimates: against greedy, its true net value ranges from +2.2% to +9.2% with perfect knowledge, −1.5% to +0.4% on the T-learner and −33.8% to −1.0% on CBR. This is the optimiser's curse [29]: the harder a planner optimises noisy estimates, the more it selects the options whose estimates happen to be too high.

Verdict ● Met. Planning by estimated effect, as the system runs by default, keeps 22% to 138% more members than contacting by risk or schedule in every setting tested, without breaking a limit. The size

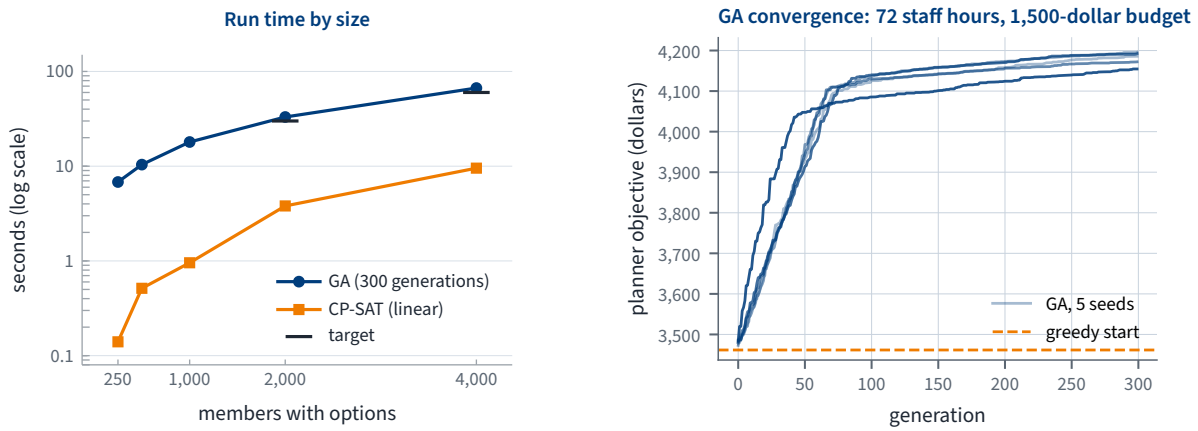


Figure 9.5 E5. Left: run time grows about linearly; this run missed the 30 s and 60 s targets by about 10%. Right: five seeds climb from the greedy start and plateau about 20% above it (regenerated on the seeded database; the committed run's five seeds end within 0.2% of each other).

Source: `results/e5_speed.csv`; `scripts/figures.py` (ga), as in `b_exact.ipynb`.

of the gain depends on the estimates more than the optimiser: on today's CBR estimates, the simpler greedy planner, selectable on the same page, keeps more members than the default genetic algorithm.

9.5 E5: How good and how fast is the genetic algorithm?

SCENARIO LAYER Benchmark: CP-SAT's proven optimum on the planner's objective

The genetic algorithm was compared with CP-SAT on the planner's own objective: the linear objective on the full gym, and the full objective (with spillover and diminishing returns) on resampled instances small enough to prove optimality.

Results. On the linear objective, CP-SAT proves the optimum for the full gym in 1.2–1.4 s; the genetic algorithm is within 0.8–2.3% of it and greedy within 15–32%. On the full objective, the genetic algorithm matches the proven optimum on 8 of 9 instances of 20–40 members and comes within 0.24% at 50; at 200 members it beats CP-SAT's time-limited best by 1.0%, though at 100 it is 2.5% below. Greedy is 14–27% below the optimum on average. Its advantage over greedy holds, at 16–26%, for every combination of spillover $\beta \in \{0, 0.5, 1\}$ and diminishing returns $d \in \{0, 0.02, 0.05\}$; replacing repair by penalties ends 2.4% lower and dropping the greedy seed 2.9% lower. It takes 18 s for 1,000 members and 33 s for 2,000 (Figure 9.5).

Verdict ● Partly met. On the linear problem CP-SAT is the better tool, exact and faster. The genetic algorithm earns its place on the full objective at real size, where it is near-optimal and robust to the assumed effect strengths; its speed is at the edge of the targets.

9.6 E6: Are the rules correct, and do guardrails matter?

SCENARIO LAYER Baseline: the same playbooks with guardrails switched off

The full rule base ran on all 4,000 members with facts built exactly as the API builds them: the validator, the 29 hand-built cases, coverage, and 60 plans with and without guardrails, scored by the true uplift.



Figure 9.6 E6. Left: of 3,448 proposals over 1,422 members, guardrails remove 1,080 (31.3%), almost all of them contacts inside the 30-day cap. Right: their cost in plan value as the scenario measures it, which has no contact-fatigue effect and so cannot see what the cap protects against.

Source: `results/e6_guardrail_bites.csv`, `results/e6_plan_value.csv`; `c_e6_rules.ipynb`.

Results. The validator reports no issues, all 29 cases match, and every rule fires for at least one member. Guardrails remove 31.3% of proposals (Figure 9.6); every guardrail-free plan contacts members again within 30 days (75–148 times per plan at 36 h), and no guarded plan contains a violation. The 386 members without a phone receive the email or in-person twin of each playbook, never an SMS or a call. At 36 staff hours, where staff time binds, removing the guardrails changes plan value by −1.0 to +1.4 members; at 120 h the guardrail-free plans gain 6–14 members by re-contacting members inside the cap. Counting contacts logged in the application doubled the proposals removed (from 15.3% to 31.3%), so next week's plan cannot contact this week's members again.

Verdict ● Met. The rule base is consistent and its guardrails do real work. Their cost is zero to small at the seeded staff level and up to about 14 members at 120 h, but the scenario has no fatigue effect to price, so we claim safety and auditability, not a retention gain. Two guardrails (G02, G03) remove nothing on today's data and remain as safety nets.

9.7 E7: Do knowledge-graph signals add information?

SCENARIO LAYER Baseline: churn probability alone

The graph's event flags were tested against the scenario's true post-event hazard (the Kaggle label knows nothing of the synthetic events): their lift, whether adding them to the churn ranking surfaces higher-hazard members among the 800 test members (1,000 bootstrap resamples), and their effect on rules and plans.

Results. The flags pick out the members the events hurt: their hazard rises 1.80 times after an anchor instructor leaves and 1.32 times after a class is cancelled, exactly the scenario's designed effects. They do not improve the ranking: every difference has an interval that includes zero (Figure 9.7), because only about 8% of test members are flagged and the churn model already explains most of the hazard. They do change who is contacted: 243 low-risk members whose routine was just disrupted, a group the churn score misses, gain an allowed free check-in. They hold 57% of the event-driven excess hazard, and at 120 staff hours contacting them adds about 7 members (+17%); at the seeded 36 hours the plan is already full and they add nothing.

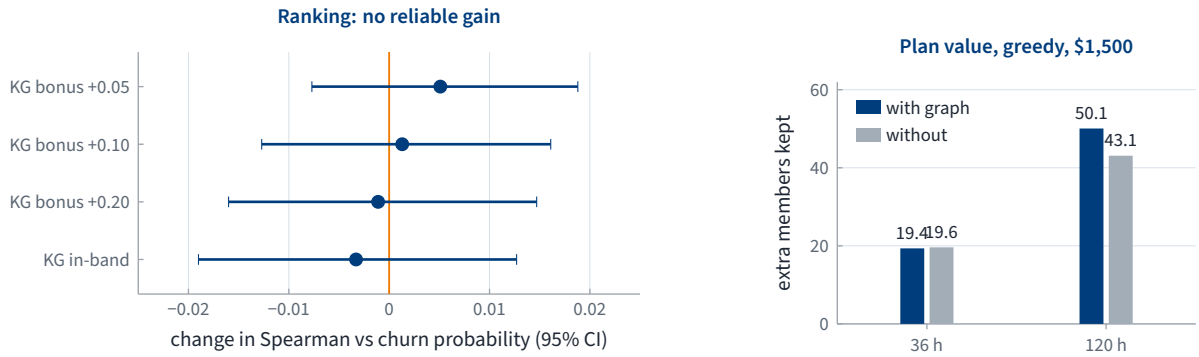
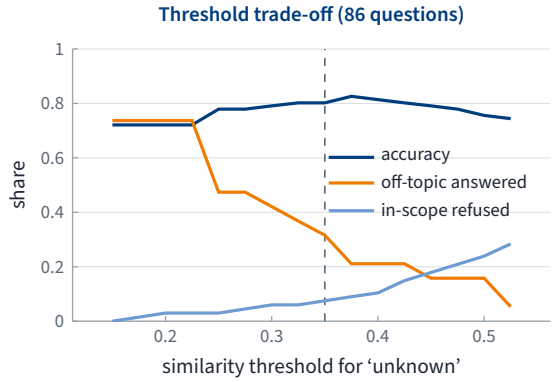


Figure 9.7 E7. Left: adding the graph's event flags to the churn ranking changes Spearman correlation with the true hazard by less than its uncertainty, whatever the bonus. Right: plan value of the greedy planner, the best in every setting, with and without the graph facts; the gain appears only when staff time is spare.

Source: `results/e7_ranking.csv`, `results/e7_plan_value.csv`; `c_e7_kg.ipynb`.



Questions	n	Classifier	Keywords
AI-written	62	**91.9**	77.4
Held out, member 1	9	55.6	**77.8**
Held out, member 2	15	**46.7**	40.0
All held out	24	50.0	**54.2**

Intent accuracy (%).

Figure 9.8 E8. Left: raising the threshold refuses more off-topic questions but also more in-scope ones; at the chosen 0.35, 32% of off-topic questions are still answered. Right: on held-out questions neither router is reliable, and the classifier is no better than keywords.

Source: `results/e8_threshold_sweep.csv`, `results/e8_intent_predictions.csv` (rerun of part 1 on 23 October with the second held-out set); `c_e8_assistant.ipynb`.

Verdict ● Partly met. The graph carries information the model lacks when events of this kind happen, and turns it into contacts the churn score would miss. That becomes retention only when staff time allows, and the evidence is a recovery of a designed effect, not of real churn.

9.8 E8: Is the assistant accurate and grounded?

SCENARIO LAYER Baseline: a keyword router with the same slot extraction

Intent accuracy was measured on 86 labelled questions: 62 written by an AI model from the intent descriptions, and 24 held out, written by two team members who never saw the training examples. Twenty sampled questions ran through the full pipeline; each sentence of the 17 model-phrased answers was labelled supported, unsupported or contradicted by an LLM judge [30], after calibrating it on known good and bad answers, and 15 answers were spot-checked by hand.

Results. Grounding is strong. No answer contained a number or member identifier absent from its evidence; 96.2% of the 80 judged sentences were supported and none contradicted; 15 of 17 answers were fully supported, and 13 of 15 were judged faithful by hand. The two failures are advice the question

asked for and a ranking the tool does not compute. A stricter prompt, adopted after a controlled replay, removed uncited sentences entirely and raised judged support from 89% to 97% at the cost of shorter answers. A local `qwen2.5:7b` model was as well supported as the hosted model (95.1% each) and far faster on a strong GPU (median 0.23 s against 4.6 s), but passed fewer format checks (71% against 88%) and gave less helpful lists. Intent routing is the weak point (Figure 9.8): the classifier is near 92% on AI-written questions but 50% on held-out ones, against 54% for keywords, mostly because it answers off-topic questions instead of refusing.

Verdict ● **Partly met**. Answers are faithful to the evidence they are given, but the router that chooses that evidence is no better than keywords on unseen questions, so we do not claim the classifier is an improvement. Samples are small, the judge varies by about two points between runs, and the spot check was assisted by an AI tool, so it is not an independent human rating.

9.9 Summary of findings

Real data supports a useful, calibrated churn model and stable, interpretable segments (E1, E2). Effect-based planning keeps substantially more members than risk-based or calendar contact (E4), but how much more depends on the quality of the uplift estimates rather than the optimiser: CBR matches the T-learner overall and explains itself (E3), yet noisy estimates turn the genetic algorithm's stronger search against it (E4), even though that search is near-optimal on its own objective (E5). The rules and graph do what they were designed to do (E6, E7), and their value shows when staff time allows. The assistant is faithful to its evidence but routes unseen questions no better than keywords (E8).

10 Findings, discussion and future work

10.1 What we built and what we found

AnchorAI turns a gym's weekly retention question into an approved, explained call list. It combines a calibrated churn ensemble, a forward-chaining rule base, a case-based uplift estimator, a genetic algorithm benchmarked against an exact solver, a knowledge graph and a grounded assistant, in one application that three roles use for the whole weekly cycle. Five findings stand out.

1. **Real data supports a useful, calibrated churn model.** On unseen members the ensemble beats a single logistic regression and a visit rule on every metric, and each risk band churns at about its predicted rate, so the planner can use the probabilities at face value (E2).
2. **Most churn sits in one recognisable group.** The 728 'new and slipping' members, under a month in and visiting about half as much as usual, account for about two-thirds of the members who left (689 of 1,061), and the trend signal the proposal anticipated holds on real members (E1, Figure 3.1).
3. **Planning by effect beats planning by risk, and estimate quality matters more than search.** Effect-based plans keep 22–138% more members than risk-based or calendar contact, but on noisy estimates a stronger optimiser selects the options whose estimates are too high, so greedy beats the genetic algorithm in truth even though the latter is near-optimal on its own objective (E3–E5).
4. **Knowledge and rules add safety and reach rather than accuracy.** Guardrails remove a third of proposals, nearly all repeat contacts, at little cost where staff time binds; the graph reaches 243 low-risk members whose routine was just disrupted, which pays off only when staff time allows (E6, E7).
5. **Grounding works; routing is the weak link.** The assistant states only what its evidence supports, but on questions its authors never saw it chooses the right tool no more often than a keyword router (E8).

10.2 Limitations

- **Synthetic ground truth.** Uplift, events and the contact history come from a documented scenario layer whose effects grow with risk by design. E3–E7 therefore show that the system recovers effects of this kind, not their real size; no real plan outcome exists.
- **One real snapshot.** The Kaggle data has no dates, so there is no out-of-time validation, recency or regularity feature, or survival model, and whether the strongest same-month columns leak cannot be settled from the data; both model versions are reported.
- **No real users.** The interviews that shaped the rules were simulated, and usability was assessed by design and by the assistant's evaluation, not with gym staff.
- **Small evaluation samples.** The assistant's held-out set has 24 questions and its faithfulness 17 judged answers; the medium risk band and the over-31 churners are small groups.
- **Untestable assumptions.** The scenario has no contact fatigue and no spillover, so it cannot reward the guardrail that prevents the one or the planning term that models the other.

10.3 Recommendations for an operator

Act on new members whose visits fall, since they are most of the churn and the driver chart shows why each is flagged. Review medium-risk members aged 31 and over by hand, where the model misses most. Keep visit dates and the outcome of every retention contact. Above all, hold back a small random control group each week, so that the real effect of each offer can be measured instead of assumed; until then, prefer the greedy planner, which the evidence favours on today's estimates.

10.4 Future work

1. **Real, time-stamped data**: validate on later months, add recency and regularity features, model time to churn with survival methods, and settle the leakage question with the operator's record timing.
2. **Uplift from randomised contacts**, replacing the scenario's assumed effects, and stronger meta-learners such as the X-learner [20].
3. **Uncertainty-aware planning** that discounts noisy estimates, so that a stronger optimiser stops amplifying their errors [29].
4. **Out-of-scope detection** for the assistant, trained on real staff questions, so that it refuses what it cannot answer.
5. **A field study** with gym staff, measuring adoption, override rates and retention against a control group.
6. **Monitoring after each rebuild**, alerting when calibration or fairness drift, and a group-specific review threshold for older members.

References

[1] Health & Fitness Association. *2025 Fitness Industry Benchmarking Report*. 2025. URL: https://www.healthandfitness.org/2025-fitness-industry-benchmarking-report/.

[2] S. Sperandei, M. C. Vieira and A. C. Reis. 'Adherence to Physical Activity in an Unsupervised Setting: Explanatory Variables for High Attrition Rates among Fitness Center Members'. In: *Journal of Science and Medicine in Sport* 19.11 (2016), pp. 916–920. DOI: 10.1016/j.jsams.2015.12.522.

[3] Health & Fitness Association. *2025 Global Fitness Industry Report Shows Record Growth and What's Next for the Market*. Summary of the 2025 HFA Global Report. 18th Aug. 2025. URL: https://www.healthandfitness.org/2025-global-fitness-industry-report-shows-record-growth-and-whats-next-for-the-market/.

[4] E. Ascarza. 'Retention Futility: Targeting High-Risk Customers Might Be Ineffective'. In: *Journal of Marketing Research* 55.1 (2018), pp. 80–98. DOI: 10.1509/jmr.16.0163.

[5] Fit Tech. *Fit Tech People: Ian Mullane, Founder, Keepme*. Fit Tech, 2020 issue 2. 2020. URL: https://www.fittechglobal.com/fit-tech-features/Fit-Tech-People-Ian-Mullane-Founder-Keepme/34579.

[6] PushPress. *Pricing*. URL: https://www.pushpress.com/pricing (visited on 23/10/2026).

[7] Gymdesk. *Pricing*. URL: https://gymdesk.com/pricing (visited on 23/10/2026).

[8] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau and C. Gagné. 'DEAP: Evolutionary Algorithms Made Easy'. In: *Journal of Machine Learning Research* 13 (2012), pp. 2171–2175.

[9] L. Perron and F. Didier. *CP-SAT*. Google OR-Tools. URL: https://developers.google.com/optimization/cp/.

[10] A. Vinueza. *Gym Customers Features and Churn*. Kaggle dataset, CC BY-NC-SA 4.0. URL: https://www.kaggle.com/datasets/adrianvinueza/gym-customers-features-and-churn.

[11] D. Arthur and S. Vassilvitskii. 'k-means++: The Advantages of Careful Seeding'. In: *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*. 2007, pp. 1027–1035.

[12] P. J. Rousseeuw. 'Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis'. In: *Journal of Computational and Applied Mathematics* 20 (1987), pp. 53–65. DOI: 10.1016/0377-0427(87)90125-7.

[13] L. Hubert and P. Arabie. 'Comparing Partitions'. In: *Journal of Classification* 2.1 (1985), pp. 193–218. DOI: 10.1007/BF01908075.

[14] G. Ke et al. 'LightGBM: A Highly Efficient Gradient Boosting Decision Tree'. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[15] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer. 'SMOTE: Synthetic Minority Over-sampling Technique'. In: *Journal of Artificial Intelligence Research* 16 (2002), pp. 321–357. DOI: 10.1613/jair.953.

[16] J. Platt. 'Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods'. In: *Advances in Large Margin Classifiers*. MIT Press, 1999, pp. 61–74.

[17] A. Niculescu-Mizil and R. Caruana. 'Predicting Good Probabilities with Supervised Learning'. In: *Proceedings of the 22nd International Conference on Machine Learning*. 2005, pp. 625–632. DOI: 10.1145/1102351.1102430.

[18] S. M. Lundberg and S.-I. Lee. 'A Unified Approach to Interpreting Model Predictions'. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[19] S. M. Lundberg et al. 'From Local Explanations to Global Understanding with Explainable AI for Trees'. In: *Nature Machine Intelligence* 2.1 (2020), pp. 56–67. DOI: 10.1038/s42256-019-0138-9.

[20] S. R. Künzel, J. S. Sekhon, P. J. Bickel and B. Yu. 'Metalearners for Estimating Heterogeneous Treatment Effects Using Machine Learning'. In: *Proceedings of the National Academy of Sciences* 116.10 (2019), pp. 4156–4165. DOI: 10.1073/pnas.1804597116.

[21] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed. Pearson, 2021.

[22] N. J. Radcliffe. 'Using Control Groups to Target on Predicted Lift: Building and Assessing Uplift Models'. In: *Direct Marketing Analytics Journal* 1.3 (2007), pp. 14–21.

[23] A. Aamodt and E. Plaza. 'Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches'. In: *AI Communications* 7.1 (1994), pp. 39–59. DOI: 10.3233/AIC-1994-7104.

[24] A. Hogan et al. 'Knowledge Graphs'. In: *ACM Computing Surveys* 54.4 (2021), 71:1–71:37. DOI: 10.1145/3447772.

[25] P. Lewis et al. 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks'. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474.

[26] N. Reimers and I. Gurevych. 'Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks'. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. 2019, pp. 3982–3992. DOI: 10.18653/v1/D19-1410.

[27] T. Saito and M. Rehmsmeier. 'The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets'. In: *PLoS ONE* 10.3 (2015), e0118432. DOI: 10.1371/journal.pone.0118432.

[28] G. W. Brier. 'Verification of Forecasts Expressed in Terms of Probability'. In: *Monthly Weather Review* 78.1 (1950), pp. 1–3. DOI: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2.

[29] J. E. Smith and R. L. Winkler. 'The Optimizer's Curse: Skepticism and Postdecision Surprise in Decision Analysis'. In: *Management Science* 52.3 (2006), pp. 311–322. DOI: 10.1287/mnsc.1050.0451.

[30] L. Zheng et al. 'Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena'. In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023.

A Project proposal

The proposal submitted on 11 September 2026 is reproduced below, pages 1–3 of the original; the fourth page is blank. Where the built system departs from it, Chapter B lists the change and the reason.

FILING REF:

PROJECT PROPOSAL

Date of proposal: 11/09/2026

Project Title: AnchorAI

Group ID (As Enrolled in Canvas Class Groups): Gate 0

Group Members (name , Student ID):

- ABDULAZIZ ALI S HASANAIN (**A0350856X**)
- ASIRI MOHAMMED YAHYA A (**A0351967N**)
- BINMANEEA KHALID WALEED M (**A0355712E**)

Sponsor/Client: *(Company Name, Address and Contact Name, Email, if any)*

Name: Aramco
Address: Dhahran 31311, Kingdom of Saudi Arabia
Contact Name: Alshahrani, Homoud T
Email: homoud.alshahrani@aramco.com

Background/Aims/Objectives:

Gym members who stop attending cancel weeks or months later. That lag is what we target. The Retention Intelligence System segments members by behavior, predicts who will leave, and decides who to contact given a fixed budget and limited staff hours. Retention is the industry's biggest cost. One in three members leaves yearly, half of new joiners within six months, taking both dues and ancillary spend with them.

Current practice fails three ways:

- Calendar campaigns contact everyone, wasting budget on members who would have stayed.
- Threshold rules ignore trend. Twelve visits a month dropping to four beats a steady two, but rules treat them alike.
- Vendor scores give a number, not a decision. Staff cannot interrogate the flag, so they ignore it.

Our goals:

- Segment members interpretably using unsupervised learning.
- Predict churn with a calibrated gradient boosting and logistic regression ensemble.
- Turn predictions into a weekly action plan where every recommendation carries a challengeable justification.

FILING REF:

Market Analysis:

The vendor landscape splits into two layers. The system of record, covering membership management, booking, payments and access control, is dominated by ABC Fitness including Glofox and Trainerize, alongside Mindbody, Xplor Technologies, Perfect Gym, Virtuagym and Daxko. Above it sits a thinner intelligence layer that bolts on rather than replacing the system of record. Keepme is the clearest example, best known for its predictive Keepme Score, targeting multi site operators and pricing by custom quote rather than published rates. Glofox offers a built in "At Risk" churn report. Current buyer guides for AI in gyms place Keepme at the head of the predictive retention and churn analytics category, with Replify leading AI reception and sales.

Four gaps define our opportunity:

- Competitors score risk but do not publicly expose constrained allocation, meaning the decision of which flagged members to contact and with which offer given a fixed budget and finite staff hours.
- Vendor messaging is uniformly built around targeting high risk members, despite academic evidence that risk ranking is suboptimal.
- Scores trained on tens of millions of profiles are powerful but unauditable, and mid market operators repeatedly cite staff distrust of black box flags.
- The intelligence layer is priced for larger chains. Keepme is generally recommended only once an operator reaches 5+ locations with real lead volume and a sales team to feed, leaving the one to five club operator, the bulk of the market by count, underserved.

We position this project as an explainable, budget aware retention planner for mid market chains, differentiating on transparent decision making rather than on model accuracy, where incumbents with far larger training corpora will always win.

Sources: *Health and Fitness Association 2025 Fitness Industry Benchmarking Report; Keepme and Glofox product documentation; industry buyer guides published in 2026.*
https://www.healthandfitness.org/health-fitness-association-releases-2025-fitness-industry-benchmarking-report/
https://www.keepme.ai/blog/making-the-most-of-your-data
https://www.glofox.com/blog/how-glofoxs-ai-can-mitigate-member-churn-and-boost-retention/
https://www.replify.ai/ai-sales-service-blog/best-ai-for-gyms-complete-buyers-guide

Project Description:

The system is scoped to cover four technique groups. Each group maps to a working component:

- **Decision automation:** A forward chaining rule engine encodes retention playbooks and commercial guardrails staff can read and edit. Fuzzy inference assigns engagement tiers, avoiding threshold cliffs. A case based reasoner picks the offer by matching similar past members and their outcomes.

- **Resource optimization:** Allocating a fixed budget across members, offers and channels is a constrained assignment problem, solved with a genetic

FILING REF:

algorithm. Fitness is expected retained value net of cost, penalized for budget, staff hour, contact fatigue and inventory violations. An exact solver benchmarks small instances so we can report the optimality gap.

- **Knowledge discovery:** K means and Gaussian mixture segmentation, a calibrated churn ensemble, an uplift model for treatment effects, and class recommendations matched to member history.

- **Cognitive techniques:** A knowledge graph of members, clubs, classes, instructors and offers propagates cohort signals such as a cancelled class slot or departed trainer. A natural language assistant answers staff questions, grounded on the rule trace and graph.

Data Collection and Preparation

We will use a hybrid strategy. The public Kaggle dataset of 4,000 members benchmarks our pipeline, but it is a single month snapshot with no intervention data, so trajectory clustering and uplift modeling are impossible on it alone.

We therefore will also build a synthetic longitudinal simulator where each member is an agent with latent habit strength, motivation and social embeddedness, visiting via a process that decays during gaps and strengthens with attendance. It is calibrated to published benchmarks: roughly 66 percent annual retention, a first 90 day cliff, elevated January churn. Treatment effects are deliberately heterogeneous so the uplift model has real structure to recover. All parameters go in the appendix. Rule base and knowledge graph content come from two to four interviews with club managers or trainers, plus published effect sizes.

Dataset: Gym Customers Features and Churn, Adrian Vinueza, CC BY-NC-SA 4.0 https://www.kaggle.com/datasets/adrianvinueza/gym-customers-features-and-churn

Features are built as of a scoring date across recency, frequency, regularity, class engagement, relationship, commercial and lifecycle families. Trend and regularity carry the argument: absolute activity is a weak signal, deviation from a member's own pattern is a strong one.
Two preparation issues:

- **Leakage.** We exclude features generated by the cancellation process itself, such as freeze requests and payment method removal. These are the label in disguise, and likely explain part of the inflated accuracy in existing gym churn papers.
- **Censoring.** Members still active at window end are treated as unobserved, not silently coded as non churners.

Identifiers are pseudonymized, no health status is inferred, fairness is checked across age and gender, and contact fatigue caps are hard constraints in the optimizer.

B Mapping to MR, RS and CGS

All four technique groups the module requires are covered: decision automation, resource optimisation, knowledge discovery and cognitive systems. MR is Machine Reasoning, RS Reasoning Systems and CGS Cognitive Systems; D is the course day.

Table B.1 Techniques as built, by course and evidence.

Component	Technique as built	Course	Evidence
Data preparation	stratified split; SMOTE-NC inside folds compared with class weights; CRISP-DM	RS D2	E2
Features	trend and ratio features, scaling, leakage check	MR D4, RS D2	E2, leakage check
Segmentation	K-means, silhouette, stability (adjusted Rand index), profiles	MR D4	E1
Churn model	class-weighted logistic regression and LightGBM ensemble, Platt calibration; F1, PR-AUC, Brier	MR D2, D4	E2
Driver explanations	averaged logistic-regression terms and TreeSHAP contributions per member	MR D3, D4	E2 drivers
Scenario layer	synthetic gym and randomised history with known effects	MR D3	scenario layer
Uplift baseline	T-learner (self-researched)	MR D4	E3
Offer reasoner	case-based reasoning: retrieve, reuse, revise (shrinkage), retain	MR D2	E3
Plan optimiser	genetic algorithm with repair, elitism, spillover and diminishing returns	RS D4	E4, E5
Exact benchmark	CP-SAT constraint optimisation	MR D1, RS D1	E5
Policy evaluation	simulation-based comparison against three baselines	RS D1	E4
Rule engine	forward chaining over derive, playbook and guardrail rules; salience; traces; versions	MR D2, RS D1	E6
Knowledge graph	Neo4j and Cypher; multi-hop cohort signals; read-only parameterised queries	MR D3, RS D5	E7
Intent detection	sentence-embedding centroids with an unknown threshold; slot filling	CGS D1	E8
Grounded generation	deterministic tool, then an LLM phrases cited evidence only	CGS D1, D3	E8
Assistant evaluation	keyword baseline; static checks; calibrated LLM-as-judge; local and hosted models	CGS D2, D3	E8
Message drafts	LLM drafts with a rule check and a template fallback	CGS D1	drafts in the application
Overall	hybrid machine learning, rules, optimisation, knowledge graph and language interface with human review	RS D1, CGS D1	E4, E6, E7

Deviations from the proposal

Table B.2 Changes from the proposal and the presentation deck.

Proposal or deck	Built	Reason
A neural network picks who to contact (deck)	genetic algorithm, with CP-SAT as exact benchmark	a neural network learns; it cannot enforce budget or staff limits
Agent-based longitudinal simulator	real Kaggle members plus a thin scenario layer	lower risk; segmentation and churn stay results on real data
SMOTE to a 50/50 balance	SMOTE-NC inside folds compared with class weights; class weights kept; Platt calibration	oversampling inflates the probabilities the planner multiplies
F1 and silhouette as the metrics	F1, PR-AUC, Brier, calibration and fairness gaps	the plan consumes probabilities
Fuzzy inference for engagement tiers	not built	not taught in the three courses
K-means and Gaussian mixture	K-means only	matches the deck and MR D4
CBR and uplift as separate components	CBR is the uplift estimator; the T-learner its benchmark	both estimate the same quantity
Recency and regularity features; censoring	not built; censoring dropped	the snapshot has no visit dates and one binary label
Exclude cancellation-process fields	a with-and-without check on same-month columns	the snapshot's leakage risk is timing
Class recommendations; surrogate decision trees; graph embeddings	not built	stretch items
Interviews with managers	simulated by an AI model, and disclosed	no real operator was reached
The LLM calls tools itself	intent router before the LLM; tool calling optional	small local models are unreliable at tool calling

C Scenario layer parameters

The scenario layer attaches a synthetic gym to the 4,000 real members, with a documented ground truth (Section 3.3). Everything below is regenerated by the data build and is deterministic in the seed (42 for every reported result). Experiments that use it test whether the system recovers known structure and decides well against a known truth; they are not claims about how real members respond.

C.1 Tables and parameters

Table C.1 The data tables written by the build: the hand-over from data to the rest of the system.

Table	One row per	Main columns	Source
`members`	member	identifier, the 14 Kaggle columns, club, monthly fee, split	Kaggle + synthetic
`instructors`, `classes`	instructor, class	club, name; a class's instructor	synthetic
`attendance`	member and class	weekly visits (always above 0)	synthetic
`offers`	offer	cost, staff minutes, stock, channels	synthetic
`interventions`	past contact	week, offer, channel, treated, stayed	synthetic
`events`	gym event	type, entity, week	synthetic
`true_uplift`	member and offer	uplift	ground truth
`true_churn`	member	hazard before and after events, exposure shares	ground truth
`features`	member	the 17 model features	derived
`segments`	segment	name, description, size, churn rate, centre	model output
`member_scores`	member	segment, churn probability, band, fee, club, top drivers	model output
`uplift_tlearner`	member and offer	estimated uplift	model output

The instructors are Sam Rivera, Priya Nair and Lee Wong (C1); Maya Chen, Omar Haddad and Ana Costa (C2); and Tom Becker, Nora Ali and Ken Sato (C3). The classes are Spin, HIIT, Yoga, Pilates,

Table C.2 Generation parameters.

Parameter	Value
Scoring date	12 October 2026
Clubs	C1, C2, C3; each member assigned uniformly at random
Instructors and classes	3 instructors per club, 2 classes each: 9 instructors, 18 classes
Classes per class-going member	1 to 3, uniformly, at the member's own club; 1,611 members attend
Monthly fee	$72, $54 or $46 for 1-, 6- or 12-month contracts
History	12 weeks; 60% of members contacted once (2,400); 60% of those treated in expectation (1,416)
Event effects	1.4 (instructor left) and 0.9 (class cancelled) in log-odds per full share of visits
Seed	42

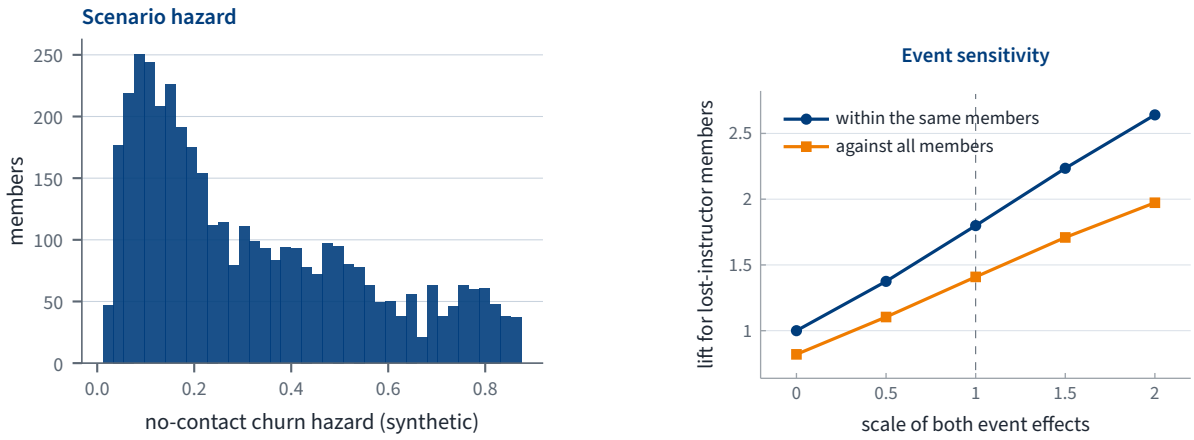


Figure C.1 Left: the synthetic no-contact hazard over the 4,000 members. Right: how the lost-instructor signal depends on the assumed event effects; at scale 0 the selection of class-goers alone puts flagged members below average (0.82). The default scale is dashed.

Source: `scripts/figures.py` (hazard); `results/a5_event_sensitivity.csv`.

Boxfit, Circuit, Zumba, Core, Stretch, Bootcamp, Barre, Step, Kettlebell, Aqua, Mobility, TRX, Dance and Power, two per instructor in that order. A class-going member's weekly visits are split across their classes with random weights so that the total equals their real Kaggle frequency.

C.2 Ground truth

Churn without contact. Each member's hazard for the next month uses features only, never the real label:

$$z = -1.4 + 1.3\,[\text{lifetime} \le 2] + 1.1\,[\text{trend} < 0.7] - 0.9\,[\text{contract} = 12] - 0.4\,\text{group} - 0.5\,(\text{visits} - 1.9), \quad \text{(C.1)}$$

with base hazard $\sigma(z)$. It ranges from 0.01 to 0.87 and correlates 0.715 with the real churn label (Figure C.1), so the scenario's risk is consistent with the real data without being fitted to it.

Offer effects. With *falling* = trend below 0.85, *new* = lifetime at most 3 months and *price-sensitive* = a monthly contract with extra spend below 120, the raw effects on next-month retention are: check-in 0.02 + 0.04 falling; free PT 0.02 + 0.14 new + 0.04 falling; class pass 0.01 + 0.10 group + 0.04 referred; discount 0.01 + 0.12 price-sensitive − 0.06 [contract = 12]. Each is multiplied by (0.6 + hazard), so persuasion matters more for members at risk, and the realised uplift saturates near certain retention: $\text{clip}(1 - h + e, 0, 1) - (1 - h)$. The discount for annual members is the only negative effect, the 'sleeping dogs' case. Over all 16,000 pairs the mean true uplift is +5.8 points; Figure 3.2 shows it by segment.

Gym events. In the week of 5 October 2026 the instructor with the most attendance (Maya Chen, Zumba and Core, C2) leaves and the busiest class at another club (HIIT, C1) is cancelled. A member's event shift is $1.4\,s_{\text{instructor}} + 0.9\,s_{\text{class}}$ in log-odds, where s are the shares of their class visits affected: a member who trained only with the departed instructor has their odds of leaving multiplied by $e^{1.4} \approx 4.1$. An event counts from its week; earlier contacts keep their pre-event outcomes. For the 186 members with at least half their visits with the departed instructor the hazard rises from 0.262 to 0.472 (×1.80); for the 170 attending the cancelled class from 0.253 to 0.334 (×1.32). Across all members it rises from 0.320 to 0.335.

Intervention history. 2,400 members were each contacted once in a random week of the 12 before the scoring date; each was treated with probability 0.6 with a uniformly random offer through a random allowed channel, and the outcome was drawn from $1 - h + u$ for treated members (with the hazard in force that week). Treated members stay 4.9 points more often than controls (Figure 3.2).

C.3 Limitations of the scenario

- **The effects are assumptions** encoding plausible playbook knowledge, not measurements; recovering them shows that a model *can* recover heterogeneous effects, not that these are the real ones.
- **Uplift grows with risk by design**, so ranking by risk within an offer already tracks the true uplift for three of four offers (Spearman 0.83, 0.87 and 0.68; 0.07 for the class pass), whereas field data shows the riskiest customers are not necessarily the most responsive [4].
- **No contact fatigue and no channel effect**, so guardrail G01 and the channel choice cannot show a benefit against this truth (E6).
- **A small, noisy history**: 2,400 yes-or-no outcomes with effects of a few points, enough to recover rankings against the truth but not to confirm them from outcomes alone (E3).
- **Only generated events are scored**; events recorded later in the application change the graph and rules but not the ground truth, and only the event effects were varied for sensitivity, not the offer effects.

D Installation guide

These steps take a fresh clone to a running, seeded AnchorAI with data, models and demo users. The data pipeline, API and web application run without Neo4j and without a language model; the graph panels and the assistant then show as unavailable.

Table D.1 Prerequisites.

Tool	Version	Used for
Git	recent	cloning
uv	0.13 or newer	the backend's Python 3.12 environment; uv downloads Python 3.12 if it is missing
Node.js and npm	Node 20 or newer	the web application
Docker with Compose	recent	Neo4j; optionally the full stack
OpenAI-compatible LLM endpoint	Ollama, OpenAI or similar	assistant and message drafts (optional)

1. Code and configuration. Clone the repository and create the environment file.

```
git clone <repo-url> AnchorAI && cd AnchorAI
cp .env.example .env
```

In `.env`, set `ANCHORAI_JWT_SECRET` to a random value (`openssl rand -hex 32`); set `LLM_BASE_URL`, `LLM_API_KEY` and `LLM_MODEL` to your endpoint (the default is a local Ollama server with `qwen2.5:7b-instruct`, installed once with `ollama pull qwen2.5:7b-instruct`); and leave `ANCHORAI_SEED` at 42 to reproduce the reported results.

2. Dataset. Download *Gym Customers Features and Churn* from Kaggle [10] and put `gym_churn_us.csv` in `SystemCode/data/raw/`. The dataset is licensed CC BY-NC-SA 4.0 and is not included in the repository.

3. Neo4j. Start the knowledge graph; its browser is at `http://localhost:7474` (user `neo4j`, password `NEO4J_PASSWORD`, `anchorai-dev` by default).

```
docker compose up -d neo4j
```

4. Backend. Install the environment and set everything up with one command.

```
cd SystemCode/backend
uv sync
uv run anchorai setup     # data build (~20 s), seeded app database, knowledge graph
```

`anchorai setup` builds the data tables and models from the raw file; creates, migrates and seeds the application database with demo users, rules, offers, roster, constraints and one approved demo week;

and loads the knowledge graph, caching its signals for the rule engine. If Neo4j is not running, the last step is skipped with a warning and can be run later with `uv run anchorai kg load`. Each step also runs on its own: `anchorai data build`, `db upgrade`, `db seed [--demo]`, `db reset` and `kg load`. The assistant downloads the `all-MiniLM-L6-v2` embedding model (about 90 MB) once, at the first start.

5. Run. Start the API and the web application in two terminals, then open `http://localhost:5173`.

```
cd SystemCode/backend  && uv run uvicorn anchorai.api.main:app --reload   # API on :8000
cd SystemCode/frontend && npm install && npm run dev                       # web on :5173
```

Table D.2 Demo users. The password of every demo user is `anchorai`, or the value of `ANCHORAI_DEMO_PASSWORD`.

User	Role	Lands on, and can use
`admin`	Admin	Dashboard; everything, including Rules, Offers & constraints, Data & models, Users, Audit log, Settings
`manager`	Manager	Dashboard; Segments, Members, Weekly plan, Scenarios, Knowledge graph, Events & roster
`staff1`, `staff2`	Staff	My contacts; Member 360, Assistant

Docker for everything. Build the data on the host first, because the demo seed needs the tables; the API container migrates and seeds the database on start. Then load the graph from the host, or with *Reload knowledge graph* on the Events & roster page.

```
cd SystemCode/backend && uv sync && uv run anchorai data build
cd ../.. && docker compose up --build -d
cd SystemCode/backend && uv run anchorai kg load
```

Checking the install. `uv run pytest -q` runs the backend tests offline in under a minute, and `npm run build` type-checks and builds the web application. In the application, the administrator's Settings page shows whether the language model, Neo4j, the models and the database are working, and tests the LLM connection.

Table D.3 Troubleshooting.

Symptom	Fix
'Expected exactly one Kaggle CSV'	put exactly one file, `gym_churn_us.csv`, in `SystemCode/data/raw/`
Pages say the data tables are not built (HTTP 503)	run `uv run anchorai data build`
Demo users cannot sign in	run `uv run anchorai db reset`
Graph panels or assistant show 'unavailable'	start Neo4j or the LLM endpoint; the rest keeps working
`npm` fails with an engine or native-binding error	upgrade to Node 20 or newer, delete `node_modules` and reinstall
Docker reports a missing host–sandbox network interface	the kernel was updated without a reboot; reboot
Port 8000 or 5173 is in use	stop the other process, or change the port

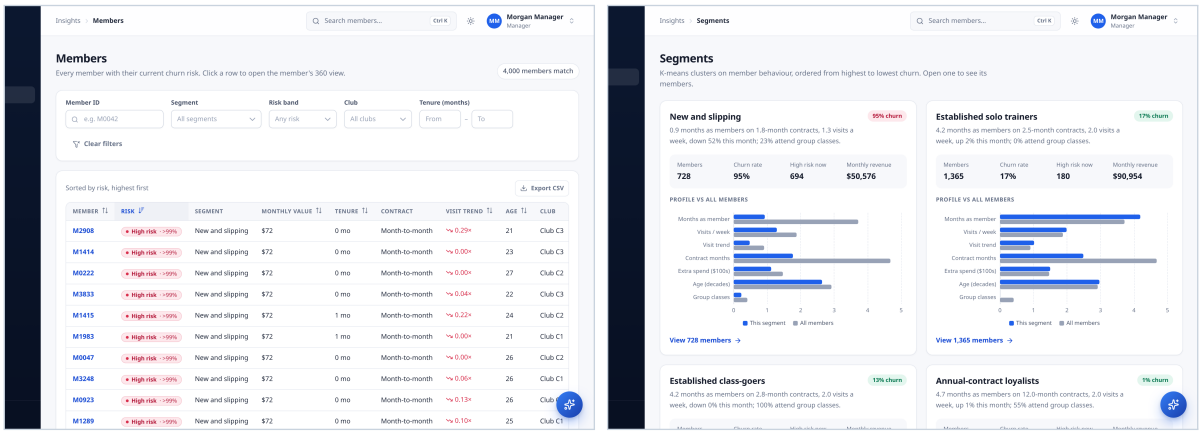
E User guide

AnchorAI has three roles, and the role decides which pages appear in the sidebar; the server checks the role again on every request (Table 8.1). Sign in at the web address with a username and password; a session lasts 12 hours. Every page has a member search at the top (Ctrl+K), a light, dark or system theme, and, except on the Assistant page itself, a round button that opens the assistant beside the current page.

E.1 Manager: the weekly cycle

Check the risk. The *Dashboard* (Figure 8.1) shows members at high risk (a churn probability of 0.6 or more), the monthly revenue at risk (the sum of churn probability × monthly fee), the share of high-risk members covered by the latest approved plan, the latest week's contacted, reached and retained counts, the risk distribution, the segment mix and outcomes by week. *Segments* shows one card per segment with its size, churn rate, members at high risk now, revenue and a profile against all members; *View members* opens the member list filtered to it. *Members* (Figure E.1a) filters by ID, segment, risk band, club and tenure, sorts by any column, keeps its filters in the page address so a list can be bookmarked, and exports every matching member to CSV. A row opens *Member 360* (Figure 8.2).

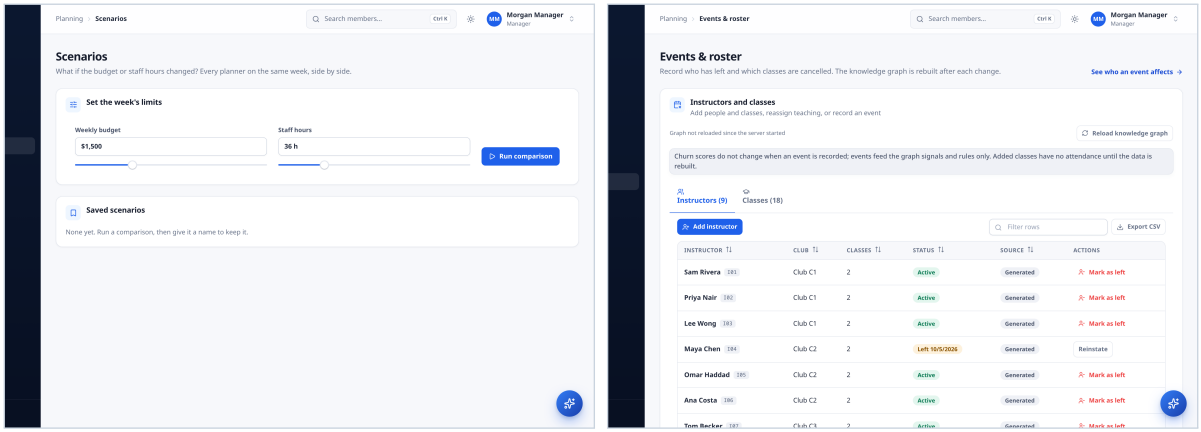
Build the plan. On *Weekly plan* (Figure 6.2), enter the weekly budget and staff hours (they start from the latest plan, or $1,500 and 36 hours), choose a planner (the genetic algorithm by default, which considers members who train together; exact; or greedy, the quickest) and select *Generate plan*. A progress bar shows each stage, and the plan finishes even if you leave the page. Four tiles sum it up: members to contact, spend, staff time and extra members expected to stay, each compared with last week.



(a) Members: every member with risk, segment, value, tenure, contract and visit trend; filters, sorting and CSV export.

(b) Segments: each segment's size, churn rate, members at high risk, revenue and profile against all members.

Figure E.1 Insight pages for managers.



(a) Scenarios: set a week's budget and staff hours and compare five planners side by side.

(b) Events & roster: record departures, cancellations and reassignments; the graph is rebuilt after each change.

Figure E.2 Planning pages for managers.

Review and change it. *Why* on a row opens the member's summary, churn drivers, the rules that fired, any gym events and the ten most similar past members with what happened to them. While the plan is a draft you can remove a member, add a contact or swap an offer; each change needs a reason and is checked against the rules at once (red: blocked, with the rule; amber: allowed but not proposed by a playbook; green: allowed and proposed). *Re-optimise the rest* keeps your changes and re-plans what is left of the budget, staff time and stock.

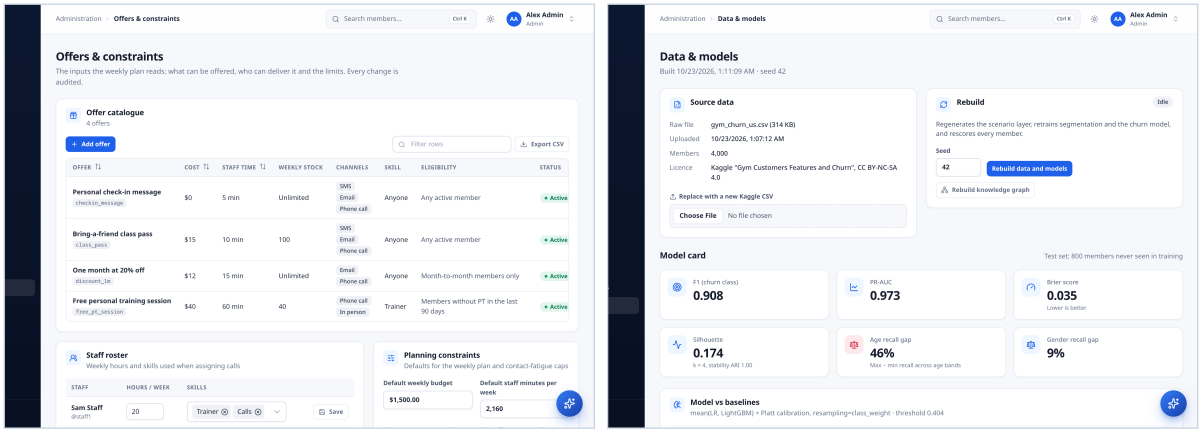
Approve and assign. *Approve plan* locks it; only one plan per week can be approved. Contacts are then shared out, most valuable first, to the staff member with the most free time who has the right skill (training sessions to trainers, calls to staff with the calls skill). Drag a contact to another staff member, re-run the assignment, and print or download each person's call sheet.

Compare what-ifs. *Scenarios* (Figure E.2a) runs five planners (contact everyone, riskiest first, greedy, exact and the genetic algorithm) on the same week at a budget and staff hours you choose, compares contacts, spend, staff time, extra members, members per $1,000 and per staff hour, and saves named comparisons. Nothing here changes the plan.

Gym events and the graph. On *Events & roster* (Figure E.2b), record what really happened: add instructors and classes, change who teaches a class, mark an instructor as left or a class as cancelled for a given week, or reinstate them. The graph is rebuilt after each change, and rules that read graph signals then fire for the affected members; churn scores do not change. On *Knowledge graph* (Figure 7.2), simulate an instructor leaving or a class closing to see who would be affected, how many are high risk and which rules would newly fire, without changing anything, and explore the neighbourhood of any member, class or instructor.

E.2 Staff: contacts, drafts and the assistant

Work the call list. *My contacts* (Figure 8.2) lists the members assigned to you in this week's approved plan, highest priority first, each with the offer and channel, why the member is on the list, talking points from their risk drivers and the rule that proposed the offer. Log an outcome in two taps: *Accepted*, *Declined*, *Reached* (no decision yet), *Follow-up* or *No answer*, optionally with a note. Accepted and



(a) Offers & constraints: catalogue, staff roster and planning constraints.

(b) Data & models: source file, rebuilds and the model card.

Figure E.3 Administration pages.

declined outcomes become new past cases, and every logged contact counts towards the contact limits, so the member is not contacted again too soon.

Draft a message. *Draft message*, next to an allowed offer in a member's rule trace, writes a short SMS, email, call script or in-person note that you edit and copy into your own tool; AnchorAI never sends anything. The tag says whether the language model or the template wrote it, and *Facts used* shows its evidence. A draft never mentions churn, risk or scores.

Ask the assistant. Type a question and press Enter (Figure 7.4). It answers about member risk, what to offer, comparisons, ranked lists, who an event affects and, for managers, the plan and what-if budgets or staff hours, always from AnchorAI's own data. Numbers in an answer are chips that show the evidence behind each sentence and open the member, rule, offer or event. When nothing supports an answer it says 'I don't know based on the data I have'. It cannot change anything, gives staff no plan details and does not remember a conversation after a refresh.

E.3 Administrator

Rules. The *Rules* page (Figure 5.2) lists every rule with its kind, salience, conditions, action and source, under a validation banner. Toggle a rule on or off, add or edit one in a form that validates as you type, or edit the YAML directly; every save needs a note and creates a new version, and nothing is overwritten. *Test on a member* runs the saved rules, or an unsaved draft, on any member and shows the same trace as Member 360. *Versions* compares any two versions line by line and rolls back to an older one, even when the active version is broken. If another administrator saved first, your save is refused and the page reloads.

Offers, staff and constraints. Edit the offer catalogue (cost, staff time, stock, channels, required skill, eligibility, active), each staff member's weekly hours and skills, and the planning defaults: weekly budget, staff minutes and the contact caps the guardrails read.

Data and models. *Data & models* (Figure E.3b) shows the source file, replaces it with a validated new Kaggle file, rebuilds data and models with any seed in the background (42 reproduces this report),

reloads the graph, and shows the model card: headline metrics on the 800 test members, the model against its baselines and against the version without suspect columns, calibration, silhouette by k and fairness by age and gender.

Users, audit and health. *Users* creates accounts, changes roles and names, deactivates users (never deletes them) and resets passwords; you cannot change your own role, and the last active administrator cannot be demoted. *Audit log* records every write with who, when, and the values before and after, filterable and exportable. *Settings* is a read-only health check of the language model (with a connection test), the knowledge graph, the models and data, and the database and planner.

F Use of AI tools

The team used an AI coding assistant (Claude Code) throughout and logged every meaningful use in the repository, with what it was used for and how the result was checked; the log is reproduced below. Three principles applied. Every output was reviewed by the member responsible, and every number in this report was re-measured against the committed results and notebooks rather than taken from the tool. Design decisions, such as the choice of estimator, planner defaults and experiment axes, were made by the team. And where AI stood in for people, it is disclosed: the manager interviews that shaped the rule base were simulated by an AI model and are treated as plausible opinion, not evidence (Section 5.3); 62 of the assistant's 86 test questions were AI-written and are reported separately from the 24 written by team members (E8). The report text was drafted with AI assistance from the team's own section drafts and reviewed by the team. No AI-generated speech is used in the videos. In the log, Abdulaziz is Abdulaziz Ali S Hasanain, Asiri is Asiri Mohammed Yahya A, and Mokdy is Binmaneea Khalid Waleed M.

Table F.1 The AI-use log, generated from `docs/AI_USE.md`; the tool in every row is Claude Code.

Date	Member	Used for	What we changed or verified
10-13	Abdulaziz	Course review, project plan, repo scaffold	Reviewed plan with team; adjusted scope
10-15	Abdulaziz	SQLite app DB, Alembic migrations, seeds, auth	Reviewed and tested
10-16	Mokdy	Member intelligence data-table contract, fixture and scenario v0 build	Reviewed; verify on real Kaggle CSV
10-16	Mokdy	Member intelligence A2–A8: features, segmentation, churn model, scenario v1, T-learner, notebooks E1/E2, member/admin APIs, web shell and pages, docs	Reviewed results against notebooks; re-measured reported numbers
10-16	Abdulaziz	Rule engine seeds, knowledge graph setup	Tested; thresholds pending
10-16	Abdulaziz	WebUI redesign	Reviewed screenshots; reviewed for regressions
10-16	Abdulaziz	Assistant chat	Reviewed and tested
10-16	Abdulaziz	Simulated manager interviews	Chose thresholds from interview and data checks; reviewed
10-17	Mokdy	Member intelligence handoffs #1–#5, #16, #18, #20: gym-event ground truth (`true_churn`), out-of-fold churn scores, signed `visit_`change, zero-visit attendance fix, phone check-ins; cross-stream review of C1–C5 (three AI review agents, each finding re-run or traced before filing); drafts of the Member intelligence report sections and user-guide pages	Re-measured every reported number against `notebooks/results/` and the model card (table cross-check script); mutation-tested the new regression tests; ran each commit's tests in isolation; browser check in both themes for all three roles; fresh-clone install test. Report and user-guide text is an AI draft for Mokdy to review and edit before submission
10-17	Abdulaziz	Review finding fixes	Reviewed and tested
10-17	Abdulaziz	Graph explorer class/instructor search + E8 prompt variant	Reviewed, queries checked on live graph

Date	Member	Used for	What we changed or verified
10-19	Asiri	Decision and optimisation handoff review: #6, #12, #13, #17, #26 replies and the #30 finding (override check misses no-phone members)	Each reply checked against the running API and the rule engine; candidate-set decision mine
10-19	Asiri	B1/B5 plan model notebook: options, objective, feasibility check, policy evaluator, baselines, first E4 grid; moved into `anchorai.optimisation` with feasibility and evaluator tests	Ran every cell and checked the outputs; tests checked by removing each limit check in turn; decisions (H = 6, baseline offer, E4 axes) mine
10-20	Asiri	Decision and optimisation walking skeleton: POST /api/plan saving plans to SQLite, and a bare Plan builder page	Checked end to end in the browser; tests for roles, feasibility and audit
10-20	Asiri	CBR uplift reasoner (B4) and E3: b_cbr notebook, three pre-registered variants, null test, plan quality vs T-learner	Ran the notebook; caught a flawed first version with the null test; decided to keep CBR per PLAN and flag the T-learner question to the team (HANDOFFS #33)
10-20	Asiri	Genetic algorithm planner (B2) with spillover and diminishing returns, and E4 on CBR, T-learner and perfect-knowledge estimates	Ran the notebook; chose the spillover design (#7); reported the optimiser's-curse finding honestly
10-20	Asiri	CP-SAT exact planner (B3), random-instance generator (B1) and E5: optimality gap, speed, seeds, sensitivity, ablations	Ran the notebook; package results checked identical to the notebook's
10-20	Asiri	Decision and optimisation backend (B6): explanations, overrides with live rule checks, approval, assignment and call sheets, my contacts, outcome logging with CBR retain, background jobs, re-optimise, saved scenarios, health checks	Tests for each rule, several checked by breaking the rule on purpose; reviewed each change
10-20	Asiri	Decision and optimisation pages (W14–W17, W20, W21, W45) and the demo seed layer (#8)	Tested every page in the browser by role, at tablet and phone sizes and in both themes; checked A's dashboard and C's assistant read the demo week
10-20	Abdulaziz	E6–E8 reruns on B's planner	Reviewed, tested, fixed what-if parsing bug found in E8
10-22	Asiri	Report sections: objectives and success measures, system architecture with two Graphviz figures, decision and optimisation design, experiments E3–E5, and the planner pages of the user guide	Directed structure and scope; reviewed each section and asked for changes; numbers checked against the results files and claims against the code; chose to cite only techniques not taught in class; references verified online
10-22	Mokdy	Member intelligence report: rewrote and extended the data preparation, knowledge discovery and E1–E2 sections; drafted the implementation section, the scenario-layer appendix and Member intelligence's input to the conclusions; corrected the installation guide (Node 20, knowledge graph under Docker); Member 360 renders Decision and optimisation's similar-cases and contact-history panels instead of raw JSON (HANDOFFS #35)	Every number checked against `notebooks/results/`, the notebooks' saved outputs and the figures; claims traced to the code; `npm run build` passes. Report text is an AI draft for Mokdy to review and edit before submission
10-23	Mokdy	Tagging the intents of the 15 E8 held-out questions (HANDOFFS #14) and adding them to `e8_questions.yaml`	Questions written by Mokdy; Claude only assigned intents and notes, wording kept as typed

Date	Member	Used for	What we changed or verified
10-23	Mokdy	LaTeX group report (`ProjectReport/latex/`): NUS-styled template, all chapters condensed from the three streams' section drafts into one voice, pgfplots charts read from `notebooks/results/`, vector figures regenerated by `scripts/figures.py`, TikZ diagrams, app screenshots captured from a local run, appendices; E8 part 1 rerun with the 15 new held-out questions	Fresh data build reproduced the model card exactly (threshold 0.404 confirmed); every decimal in the report cross-checked against results files, notebook outputs and the model card; all DOIs resolved; 195 backend tests passed. Report text is an AI draft from the team's sections, for the team to review

G Individual project reports

One page per member, with their individual reflection. Peer reviews are confidential and submitted individually.

Abdulaziz Ali S Hasanain

Individual reflection

My contribution. I helped set up the project plan, split the work into three streams with shared contracts, and scaffolded the repository. I then led the knowledge and interaction part of the system. I built the rule engine, which uses forward chaining, along with its validator, guardrails and rule versioning. I built the Neo4j knowledge graph and the cohort signals it feeds back into the rules. I also built the staff assistant, which routes a question to a tool and answers only from that tool's output. Finally, I built the app database with migrations, seeds and login by role, ran experiments E6 to E8, and did general UI/UX work in the web app.

What I learned. The most useful lesson was how the three course modules fit together in one system. From Machine Reasoning, writing rules as data with a validator taught me that a rule base needs the same care as code. It needs tests that state which rules must fire, checks for contradictions, and versions that can be rolled back. From Cognitive Systems, the knowledge graph showed me that structure can add reach rather than accuracy. It flagged 243 members at low risk whose routine had just been disrupted, members the churn model would never rank. The assistant taught me that grounding matters more than the model. Its answers were 95.1% supported by evidence with both a hosted and a local model, but its intent routing was no better than keywords on unseen questions, so the weak point was the routing and not the language model.

The hardest part was the trade off between guardrails and plan value. Guardrails removed 31.3% of proposals, almost all of them repeat contacts within 30 days. When staff time was tight this cost almost nothing, but at 120 staff hours the plans without guardrails kept 6 to 14 more members, because our scenario has no contact fatigue effect to penalise repeat contacts. An evaluation can only measure what it models, so we claim safety and auditability for the guardrails and not a retention gain. Next time I would build the cost of the behaviour a rule protects against into the evaluation from the start.

Integration was the other big lesson. Machine learning, rules, case based reasoning and optimisation each worked alone well before they worked together. Agreeing on schemas early, and keeping the system running when the LLM or the graph was unavailable, saved us more time than any single model did.

Applying it. I work as a product development engineer at LAB7, the venture building arm of Aramco, where many of the startups we build and support are developing AI systems. From this project I take a checklist for them. Keep business rules explicit and versioned beside the model, ground any LLM feature in tool output and measure its faithfulness, use a knowledge graph where relationships carry signal, and decide early how a system will be evaluated when real outcomes are not yet available. Above all, I now push for explainability as a design requirement, since staff will only act on a plan they can question.

Asiri Mohammed Yahya A

Individual reflection

To be written by Asiri Mohammed Yahya A.

Binmaneea Khalid Waleed M

Individual reflection

My contribution. I led the member intelligence part of the system. I defined the data-table contract that the other parts build against, and I built the data layer. It joins the Kaggle members with a synthetic scenario of clubs, classes, instructors, offers, past retention contacts and gym events, and it carries ground truth so that uplift and event effects can be measured. I built the feature engineering with its leakage check, the K-means segmentation (E1) and the calibrated churn model (E2). The model averages logistic regression and LightGBM, scores training members out of fold, and explains each score with per-member drivers. I also built the T-learner uplift baseline. On the app side I built the member and admin APIs, the data upload and rebuild flow, the dashboard and the member pages, including Member 360, which brings all three parts together. Finally, I reviewed the decision automation work, wrote the installation guide, and wrote the data, member intelligence and E1–E2 parts of this report.

What I learned. The most useful lesson was that a good score is not the same as a good model. The churn model reached an F1 of 0.908 against 0.549 for a simple visit rule, but much of that came from a trend signal that compares this month's visits with the member's average. Without that signal and its close relatives, F1 fell to 0.817, which is the figure I would quote to a gym that measures visits differently. Calibration mattered as much as ranking. The planner multiplies probabilities by uplift and cost, so a well-ranked but badly calibrated score would have produced plans that look optimal and are not.

The scenario layer was the hardest part. The Kaggle data has no offers, no past contacts and no gym events, so I had to generate them, and everything generated is an assumption. I kept the ground truth separate from the training data, and I recorded which effects were invented and how strong they were. Even so, the uplift baseline recovered only part of the true ranking (Spearman 0.39 overall, 0.61 for free personal training), and the small contact history could not confirm whether it or the case-based reasoner gave better plans. Uplift is much harder to learn than risk, and an honest evaluation has to show that.

The fairness check was a lesson too. Recall for members aged 31 and over was 0.52 because they rarely leave, so the same model treats age groups differently without any change to its inputs. I now check groups separately as a matter of course.

Applying it. From this project I take a habit of checking for leakage before trusting a strong result, calibrating any score that feeds a decision, and stating plainly what a synthetic evaluation can and cannot show. Agreeing on the data-table contract early also let the three parts work in parallel, and I would do that first on any team project.